Informatica 2.2 / Computational Intelligence
E5: cellular automata & agent-based modeling

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026

Università
Ca'Foscari
Venezia

Outline

- Cellular automata
- Conway's Game of Life
- Agent-based modeling

Stanisław Ulam

- Polish-American scientist working in math and nuclear physics
  - Came to US in 1935 following an invitation by Von Neumann, family members got killed in the holocaust
  - Worked with Von Neumann in the **Manhattan project**, where he originated the Ulam-Teller typical design of thermonuclear weapons
  - Invented **Monte Carlo** computation
  - Also proposed nuclear pulse propulsion

- Together with Von Neumann, he also introduced the concept of **Cellular Automata**

Cellular Automata

- A **Cellular Automaton** (CA) is a **discrete dynamic model of computation** studied within the automata theory
  - **Discrete**: all elements are **finite and numerable**
  - **Dynamic**: we start from an **initial state** and calculate the **evolution in time**

- CA are exploited in **various disciplines** including structure modeling, economy, sociology, generative art, chemistry, theoretical biology and, of course, **physics**

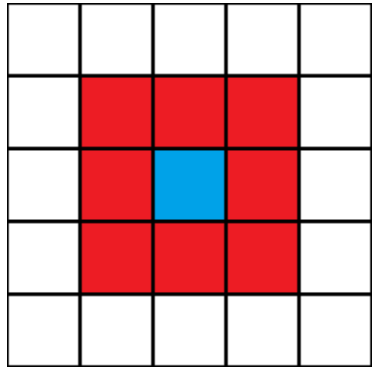
- CA are a typical topic in **complexity theory** due to their unpredictable behavior

- CA consist on **regular grids** (of any dimension) of **cells…**

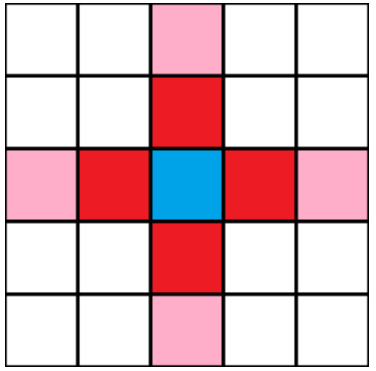
Cells

- Each **cell** can have a **state** (e.g., a binary state: on or off)

- Each cell has a **neighborhood**
  - Generally, adjacent cells within a given distance
  - Note that, in the example, both Moore's and Von Neumann's neighborhoods consider the same *number* of cells
  - Alternative topologies are possible (e.g., hexagonal)



Moore neighborhood



Von Neumann neighborhood

- Cells **change their state** according to **rules** (i.e., functions) which are calculated according to the **states of the cell itself** and the **state of its neighborhood**

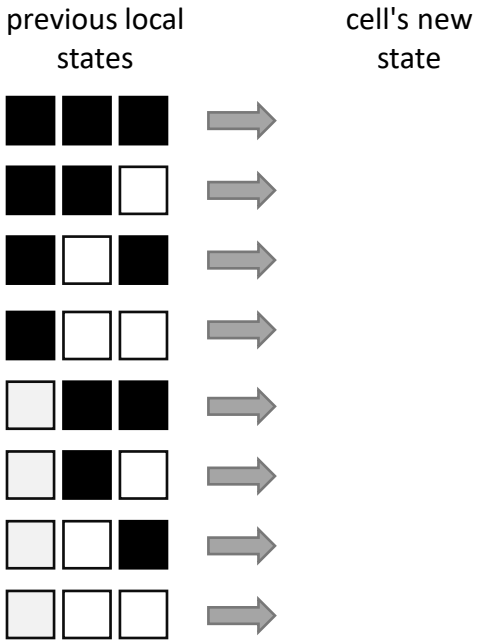
About rules

- Rules describe **local** **updates of state**
  - That is, how to **map the state of a cell**, along with the **state of its neighbors**, to a **new state**
  - Nothing else can influence the state of the cell
  - **The global behavior of the automaton *emerges*** from the local behavior of cells

- As an example, let's assume a **binary 1D CA**
  - For each cell, assume that the neighborhood is **one cell**, i.e., the cell on the **left** and the cell on the **right**
  - For simplicity, let's denote **state 0 with white and state 1 with black**
  - **How many rules can we define for such an automaton?**

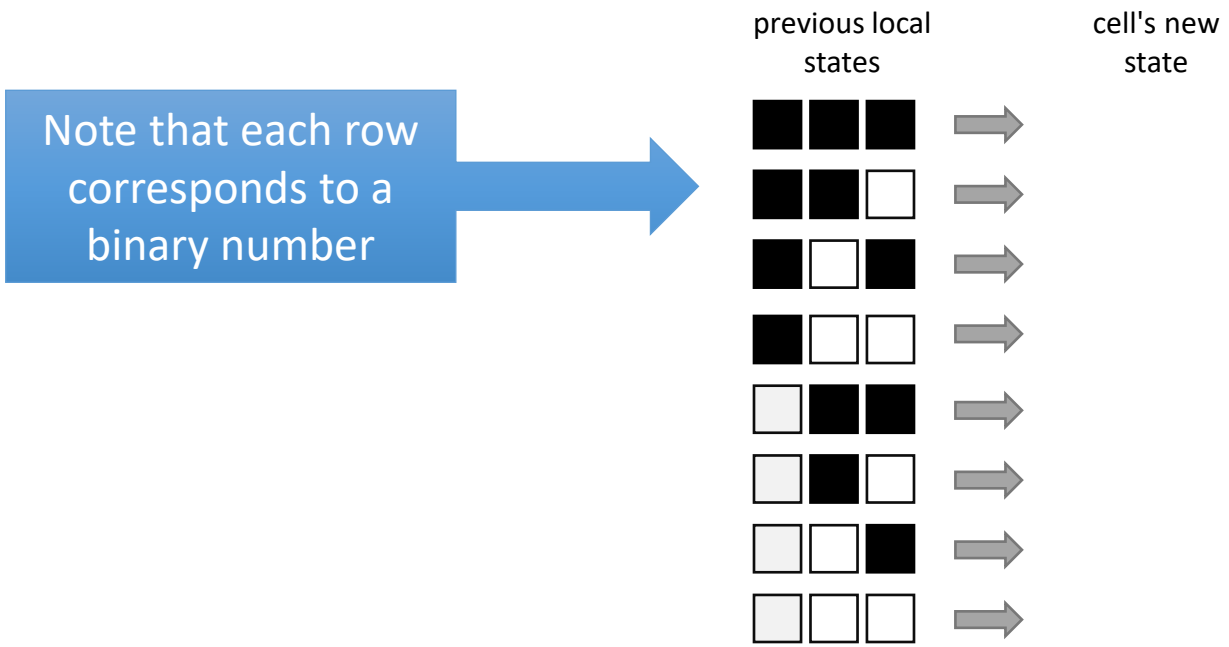
How many rules?

- In the case of a 1D binary automaton, with neighborhood η cells, the number of possible rules is $\gamma = 2^{2\eta+1}$
  - For instance, in the case of $\eta = 1$ we obtain $2^3 = 8$ possible rules to be defined



How many rules?

- In the case of a 1D binary automaton, with neighborhood η cells, the number of possible rules is $\gamma = 2^{2\eta+1}$
  - For instance, in the case of $\eta = 1$ we obtain $2^3 = 8$ possible rules to be defined

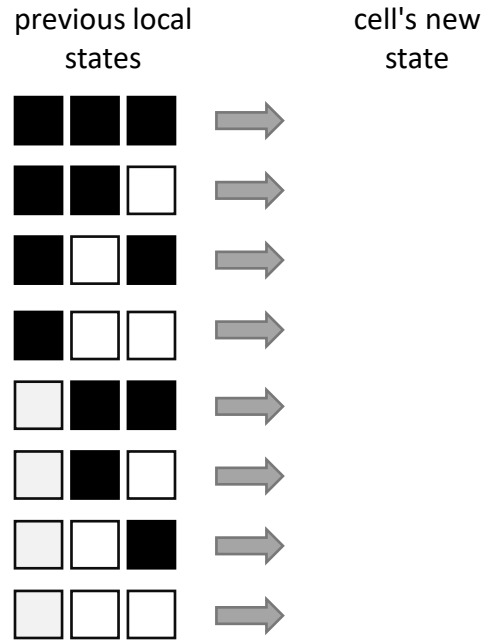


How many rules?

- In the case of a 1D binary automaton, with neighborhood η cells, the number of possible rules is $\gamma = 2^{2\eta+1}$
  - For instance, in the case of $\eta = 1$ we obtain $2^3 = 8$ possible rules to be defined

Note that each row corresponds to a binary number

Also note that in a binary automaton the new state can only be either 1 or 0



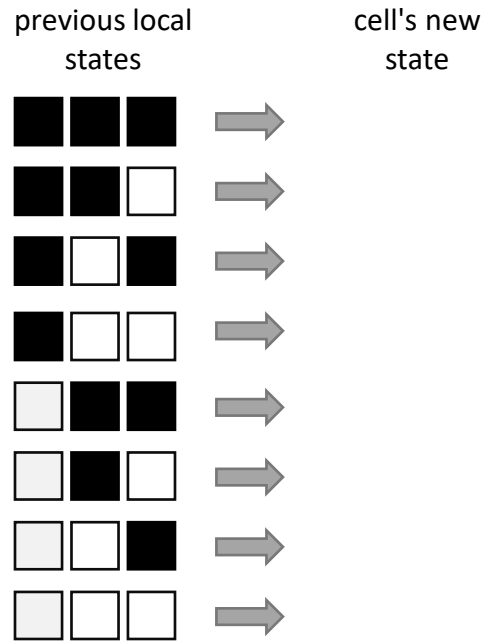
How many rules?

- In the case of a 1D binary automaton, with neighborhood η cells, the number of possible rules is $\gamma = 2^{2\eta+1}$
  - For instance, in the case of $\eta = 1$ we obtain $2^3 = 8$ possible rules to be defined

Note that each row corresponds to a binary number

Also note that in a binary automaton the new state can only be either 1 or 0

Thus, there are $2^{\gamma} = 256$ possible CA that can be defined in this case



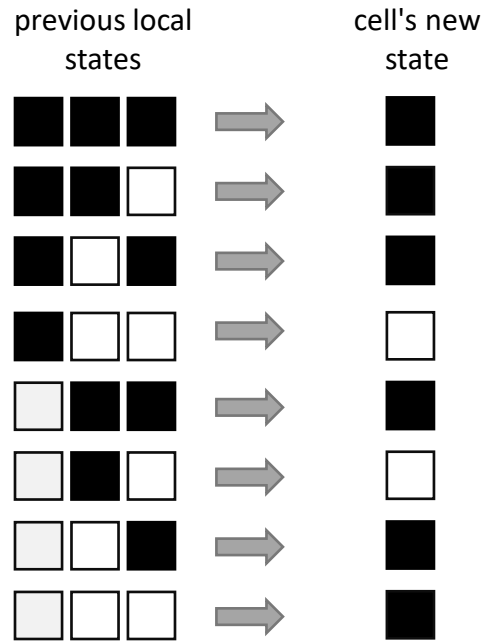
How many rules?

- In the case of a 1D binary automaton, with neighborhood η cells, the number of possible rules is $\gamma = 2^{2\eta+1}$
  - For instance, in the case of $\eta = 1$ we obtain $2^3 = 8$ possible rules to be defined

Note that each row corresponds to a binary number

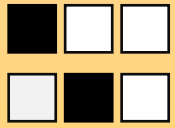
Also note that in a binary automaton the new state can only be either 1 or 0

Thus, there are $2^{\gamma} = 256$ possible CA that can be defined in this case



This is **one** possible set of rules

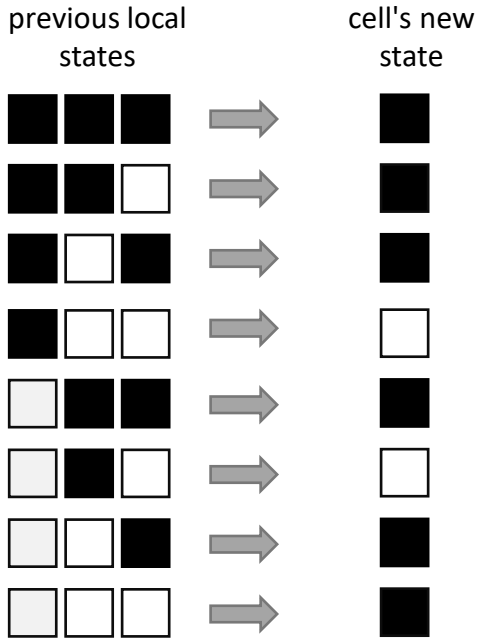
The state becomes zero only in two cases:



Indexing rules

- Given that the left part of the table basically encodes binary numbers from 0 to 7, we can focus our attention on the **right column**

- That is a binary representation too
  - For instance, in this example would be $1110\ 1011$
  - In decimal numbers, $1110\ 1011_2 = 235_{10}$
  - Thus, this would be **rule 235**

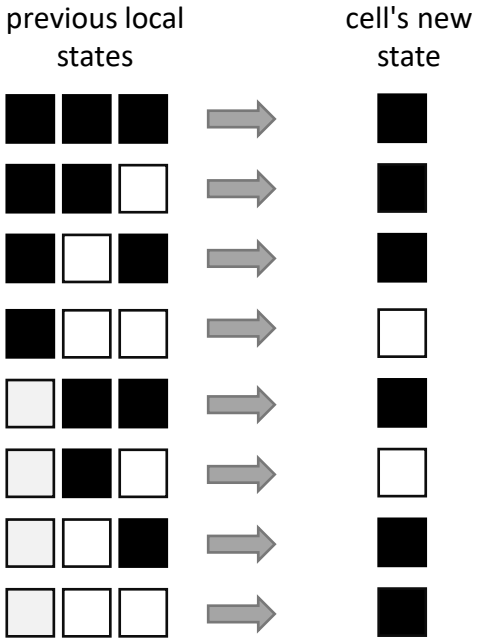


Cell state updates

- In conventional CA, the rules are applied in a massively **parallel fashion**
  - That is, there is a global "clock" and all states are updated **synchronously**
  - That is, a state update at time t only affects the neighbouring automata at time $t + 1$

- There exist **many alternative CA models** (e.g., 3D, async, non-binary, probabilistic) but we will not study them here

- **Let's now simulate** our Rule 235 and see what it does https://devinacker.github.io/celldemo/



Uhm

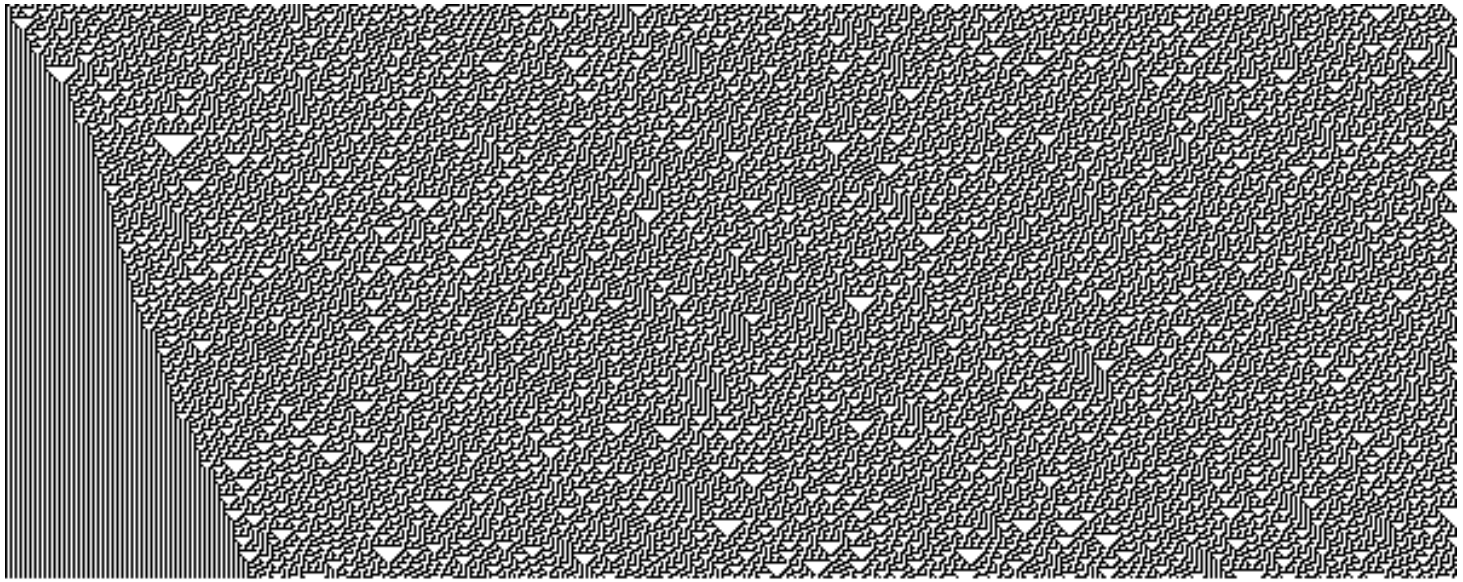
Initial (random) state

Time

Rule 30

- Rule 235 was not that interesting. Let's try another one.
- Let's check **Rule 30**

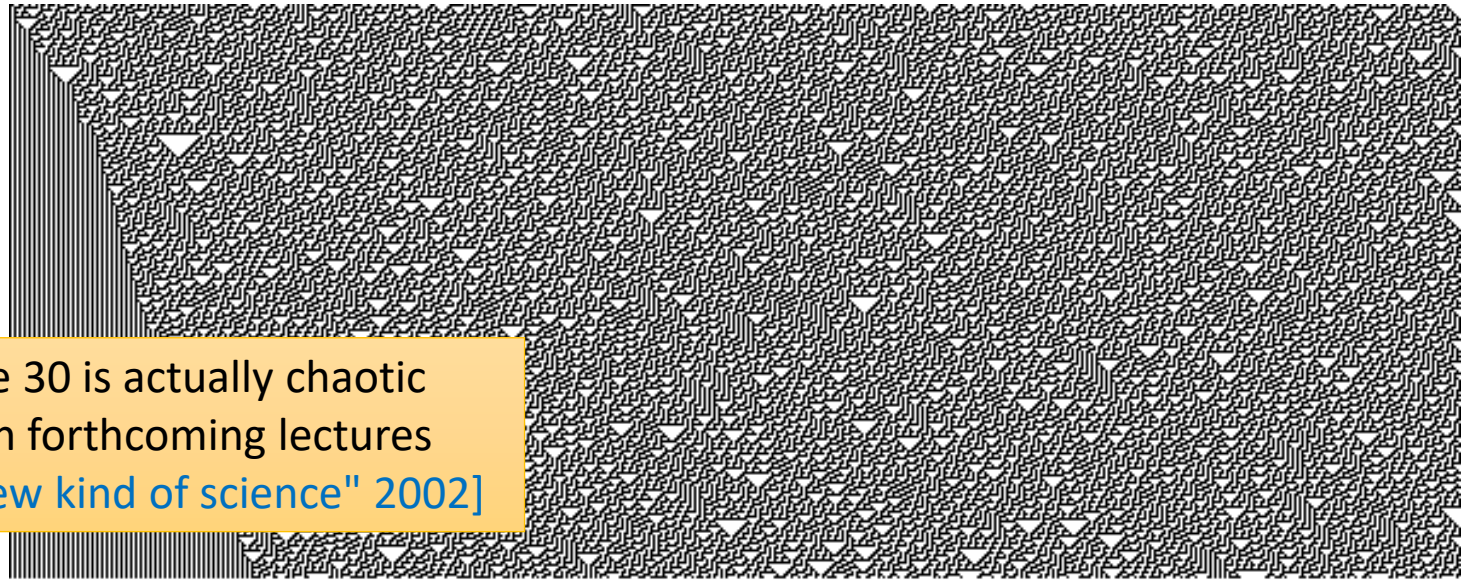
This is DEFINITELY more interesting: it is not periodic nor regular



Rule 30

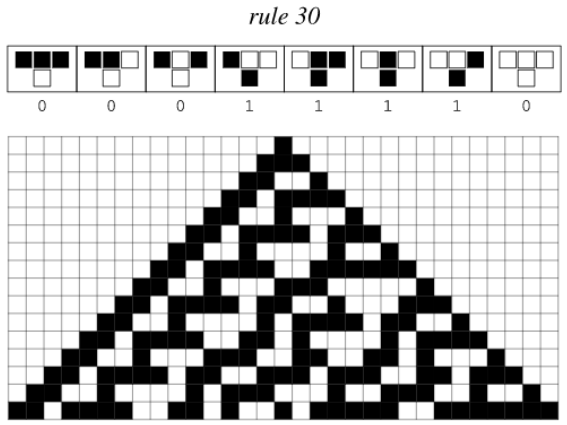
- Rule 235 was not that interesting. Let's try another one.
- Let's check **Rule 30**

This is DEFINITELY more interesting: it is not periodic nor regular



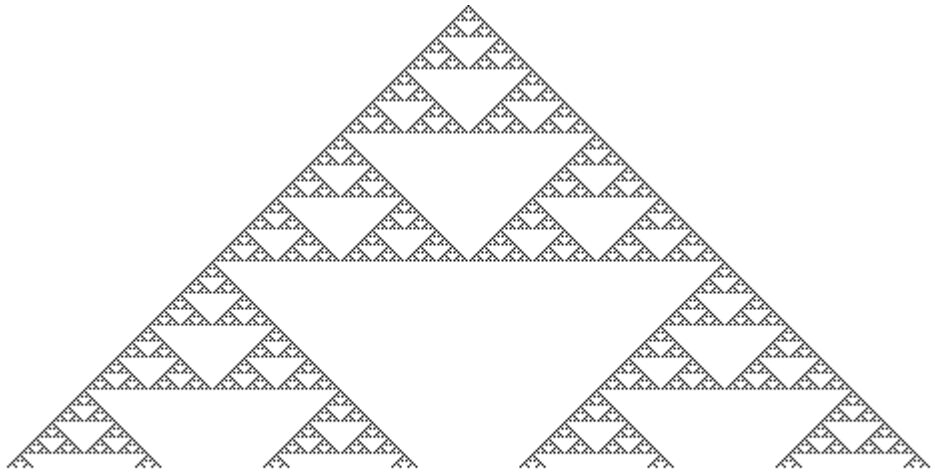
It turns out that rule 30 is actually chaotic
More about Chaos in forthcoming lectures
Check: [Wolfram, "A new kind of science" 2002]

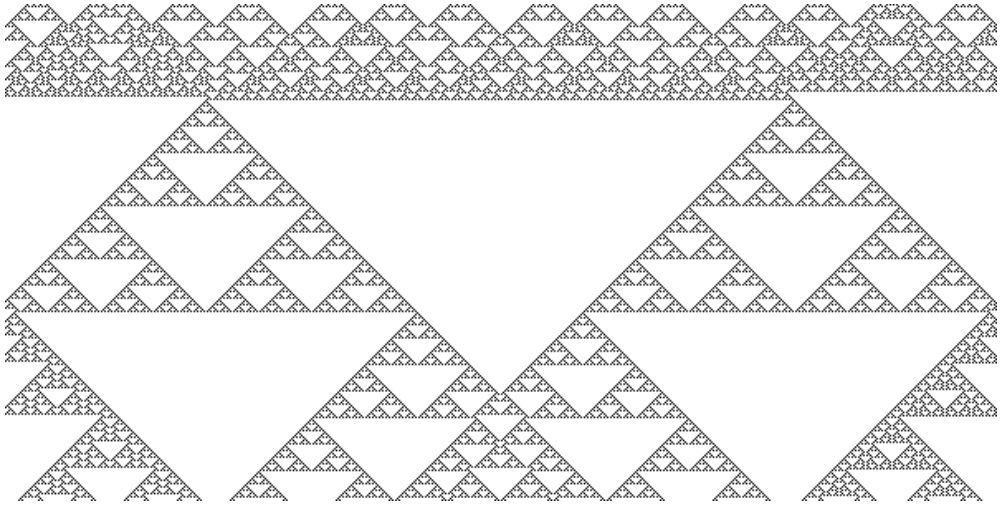
Rule 30 in *Nature*



Let's try again. Rule 90

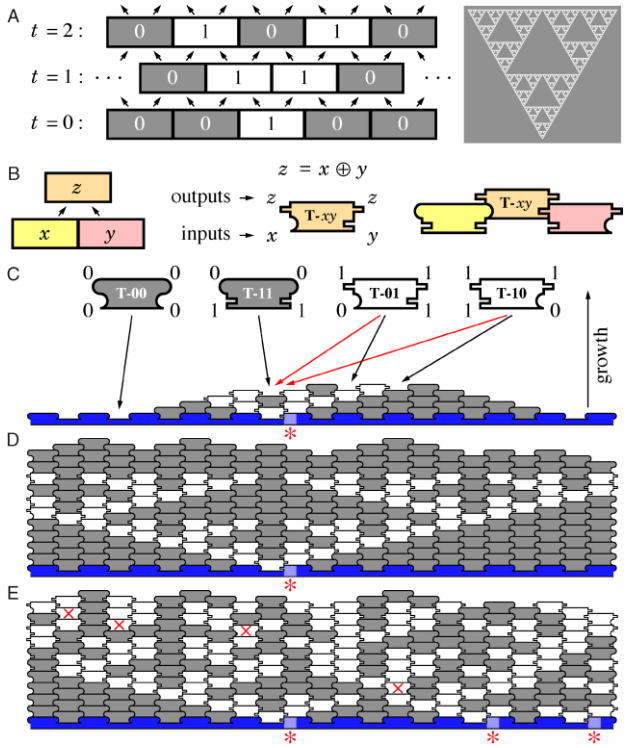
- If started from a single cell, the automaton produces a **fractal** structure named the **Sierpinsky triangle**

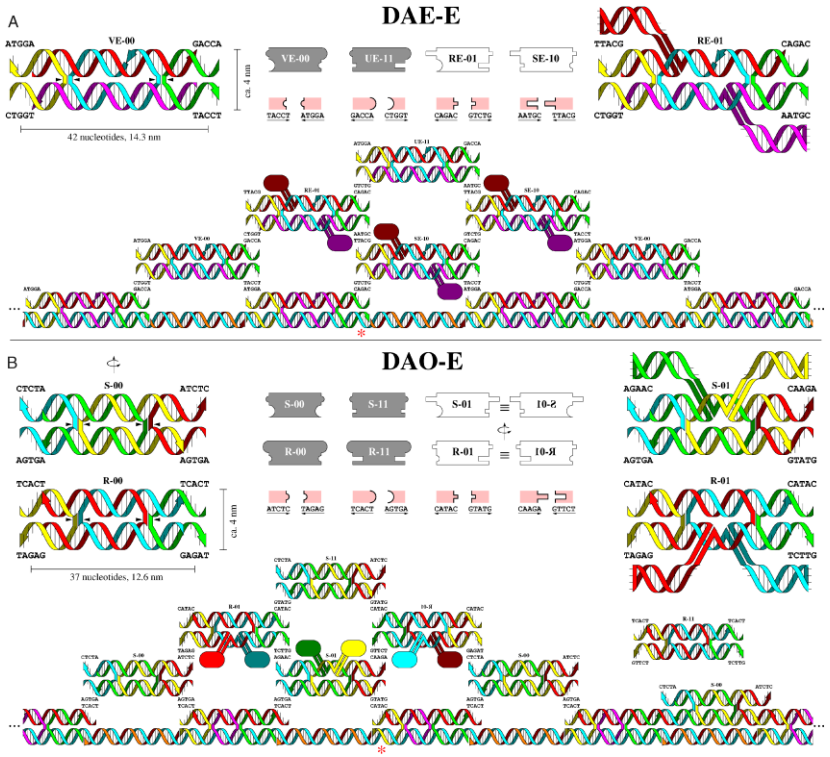


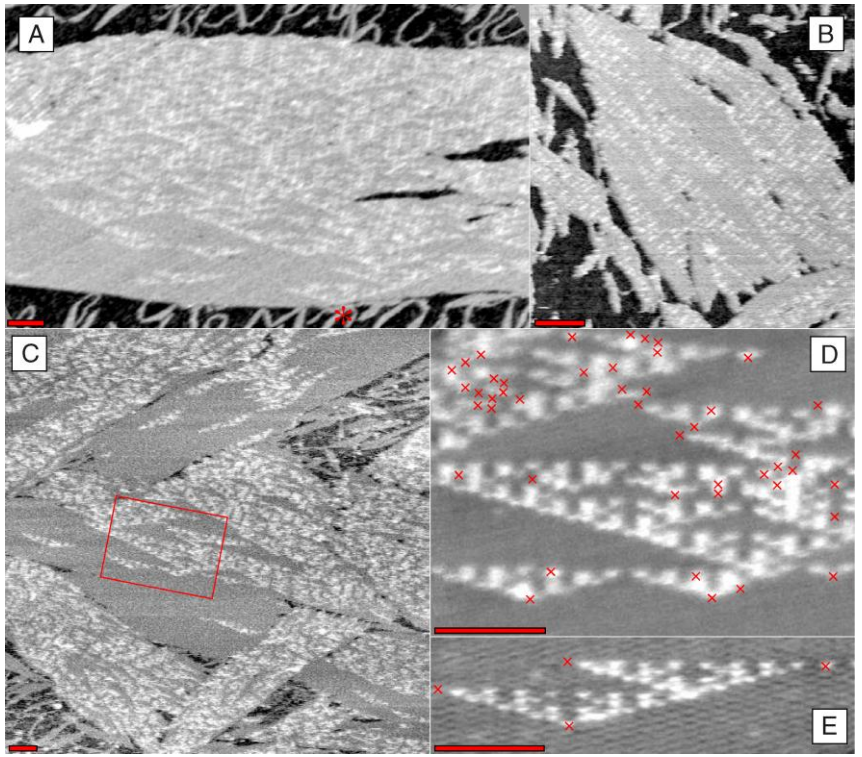


Rule 90 in nature (?)

- Rothemund, Papadakis and Winfree created a Rule 90 on a nano-scale by means of **self-assemblying DNA** (more about this in the lecture on **DNA computing**)
  - [Rothemund et al., Plos Biology 2004]







A new kind of science

- Wolfram investigated the dynamics of the CA rules and determined that there exist **four classes**:
  - Class 1: **uniformity**. Rules leading to an **homogeneous state**, the CA no longer change its cells
  - Class 2: **repetition**. Rules leading to stable (not constant) **periodic patterns**
  - Class 3: (pseudo)**randomness**. Rules leading to **irregular states**
  - Class 4: **complexity**. Rules leading to **unpredictable or chaotic rules**

Higher dimensions: the Game of Life

- [Conway, 1970]
- **2D binary CA** with 8 neighbors
- Cells denote individuals in a population (1=alive, 0=dead)
- 4 simple rules:

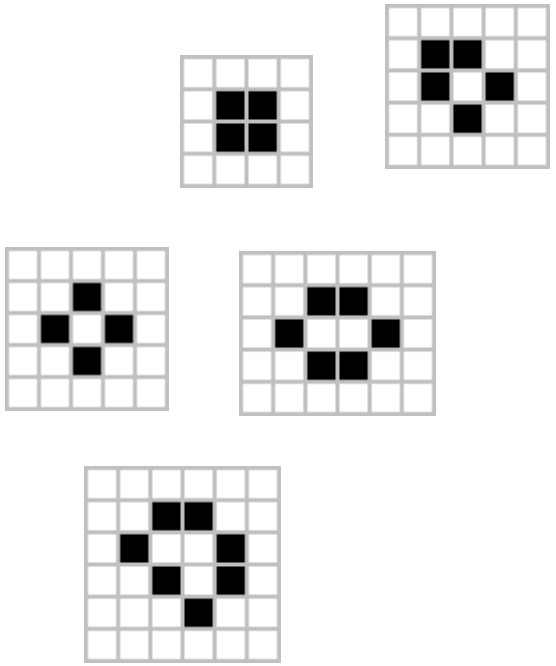
Reproduction	Survival	Death by underpopulation	Death by overpopulation
Any **dead** cell with **exactly three live neighbours** becomes a **live cell**, as if by reproduction	Any **live** cell **with two or three live neighbours lives** on to the next generation	Any **live** cell **with less than two live neighbours dies**, as if by underpopulation	Any **live** cell with **more than three live neighbours dies**, as if by overpopulation

https://playgameoflife.com/

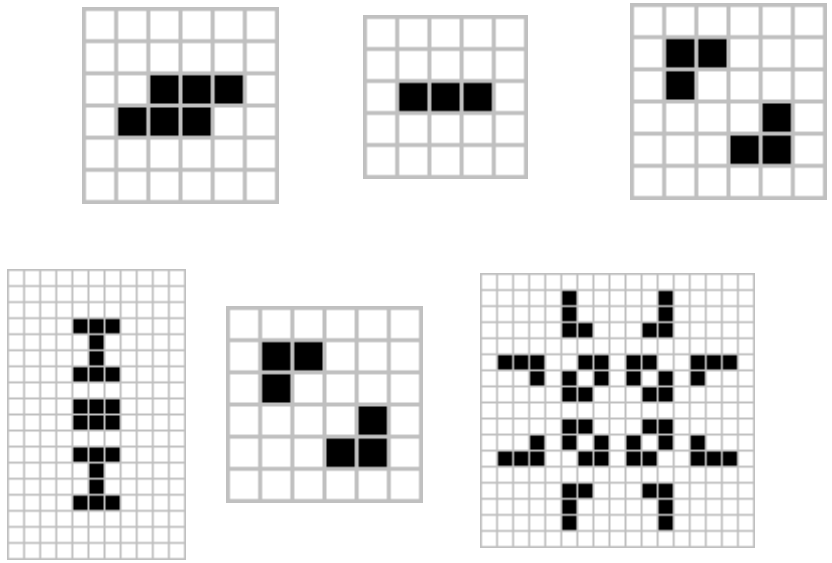
Dynamic evolution of the GoL

- **Several patterns can emerge** from these simple four rules and can are usually categorized into **three classes**

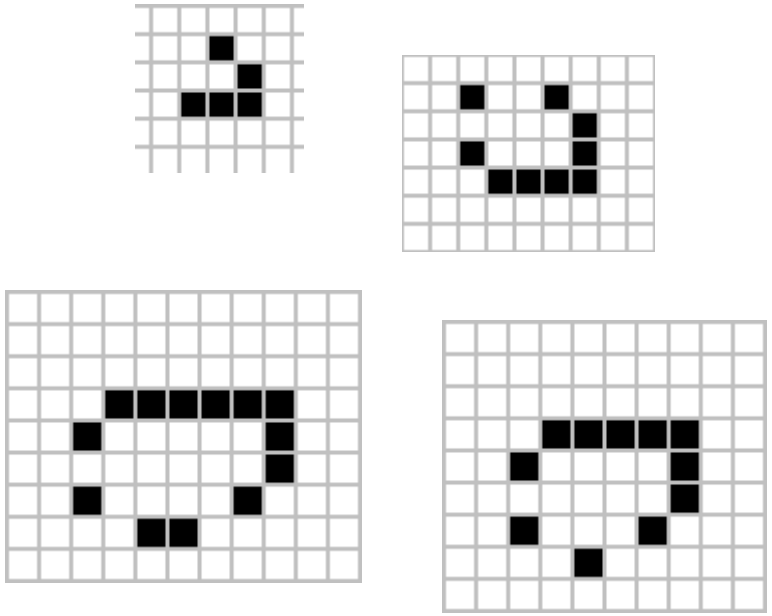
Still lifes



Oscillators

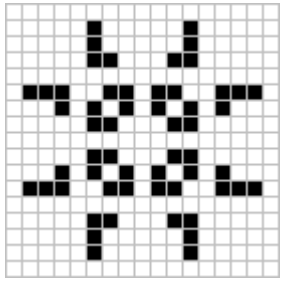


Spaceships

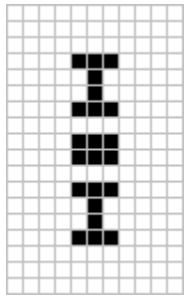


Non-trivial patterns

- The "Pulsar" is the most common period-3 oscillator



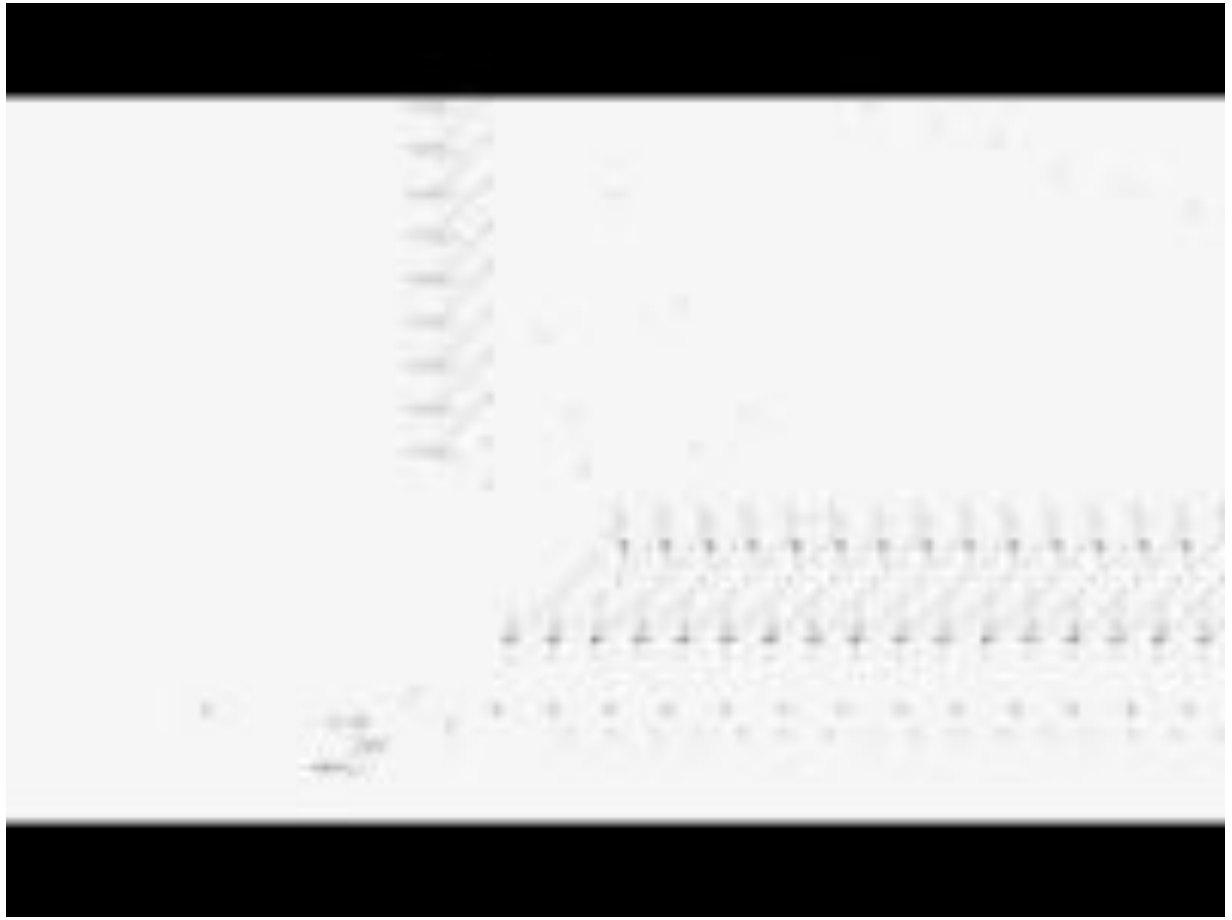
- The Penta-decathlon has period 15



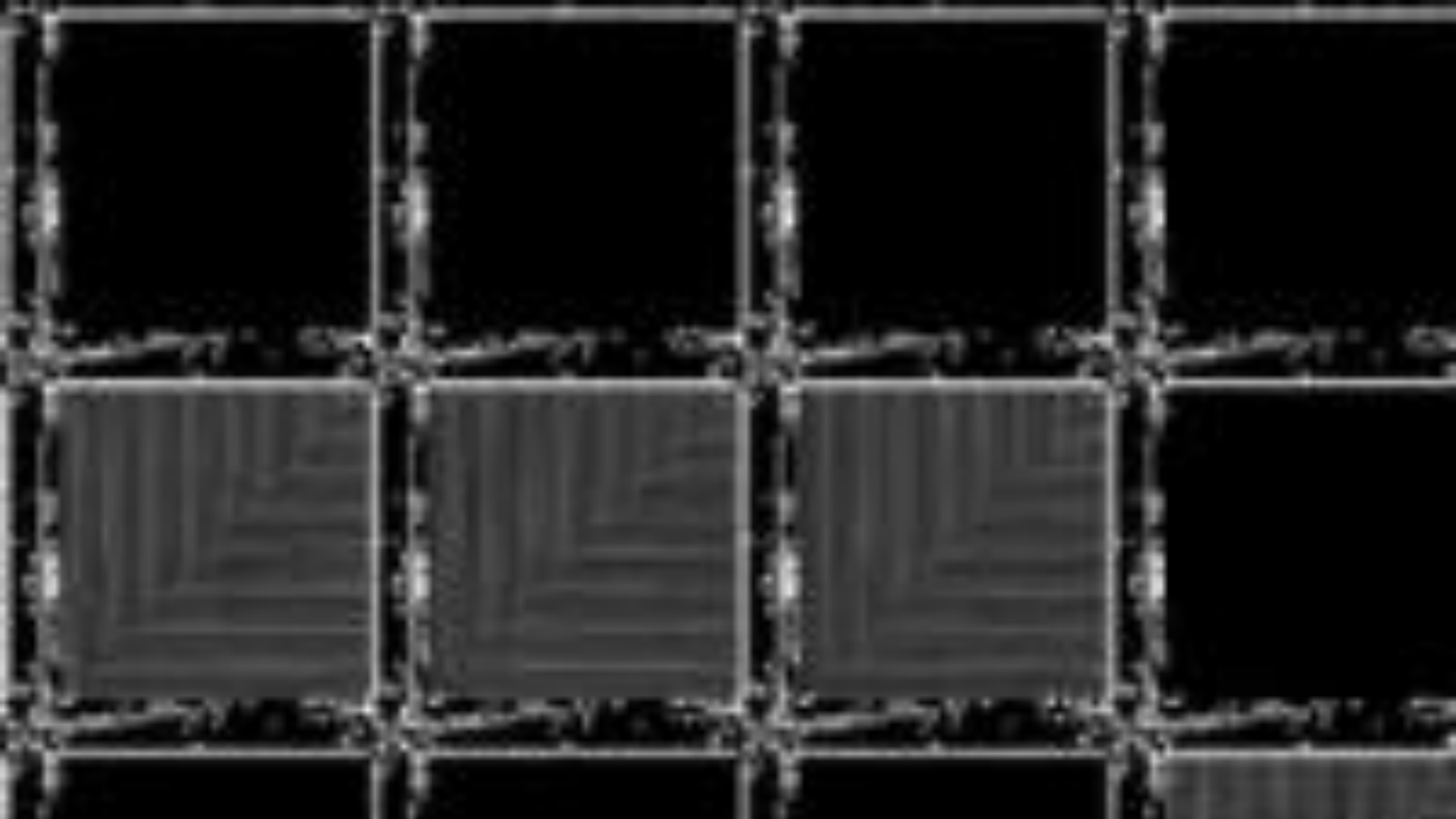
- Additional **rare oscillators** of period 4, 8, 14, and 30 arised from random initial conditions. Afaik, none *ever* witnessed period-19, -34 and -41 oscillators!

- Some patterns implement **logic gates**: GoL is **Turing Complete**!

Twin prime calculator with GoL

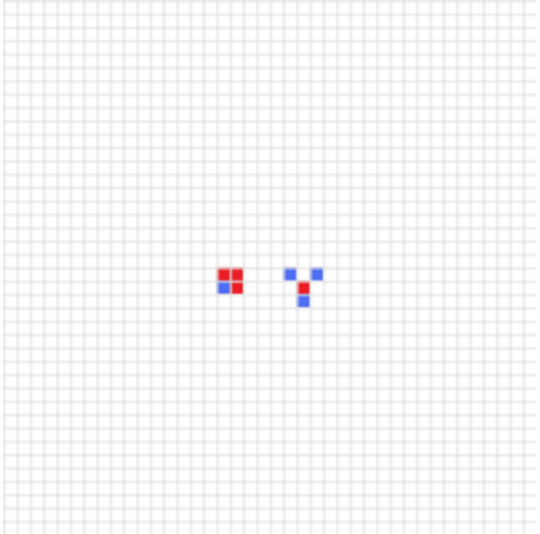


https://www.youtube.com/watch?v=iNB-FLOJWn0



Methuselah

- A Methuselah is a **small seed pattern** of initial cells which **takes a large number of generations to stabilize**
  - [Gardner, 1983]
  - Small seed = fewer than 10 cells
  - Large number of generations = more than 50
  - Never stabilizing → not a Methuselah (e.g., oscillators)

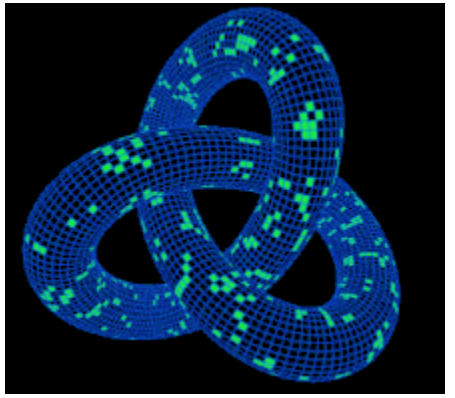


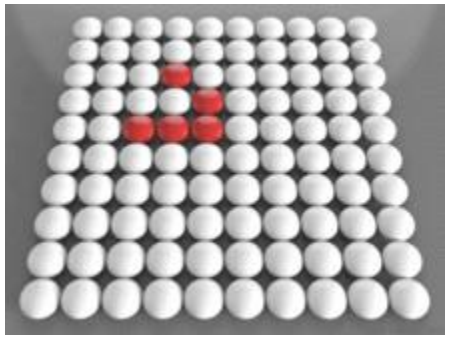
From Wikipedia: The "die hard" Methuselah lives for 130 generations before all cells die

Boundary conditions

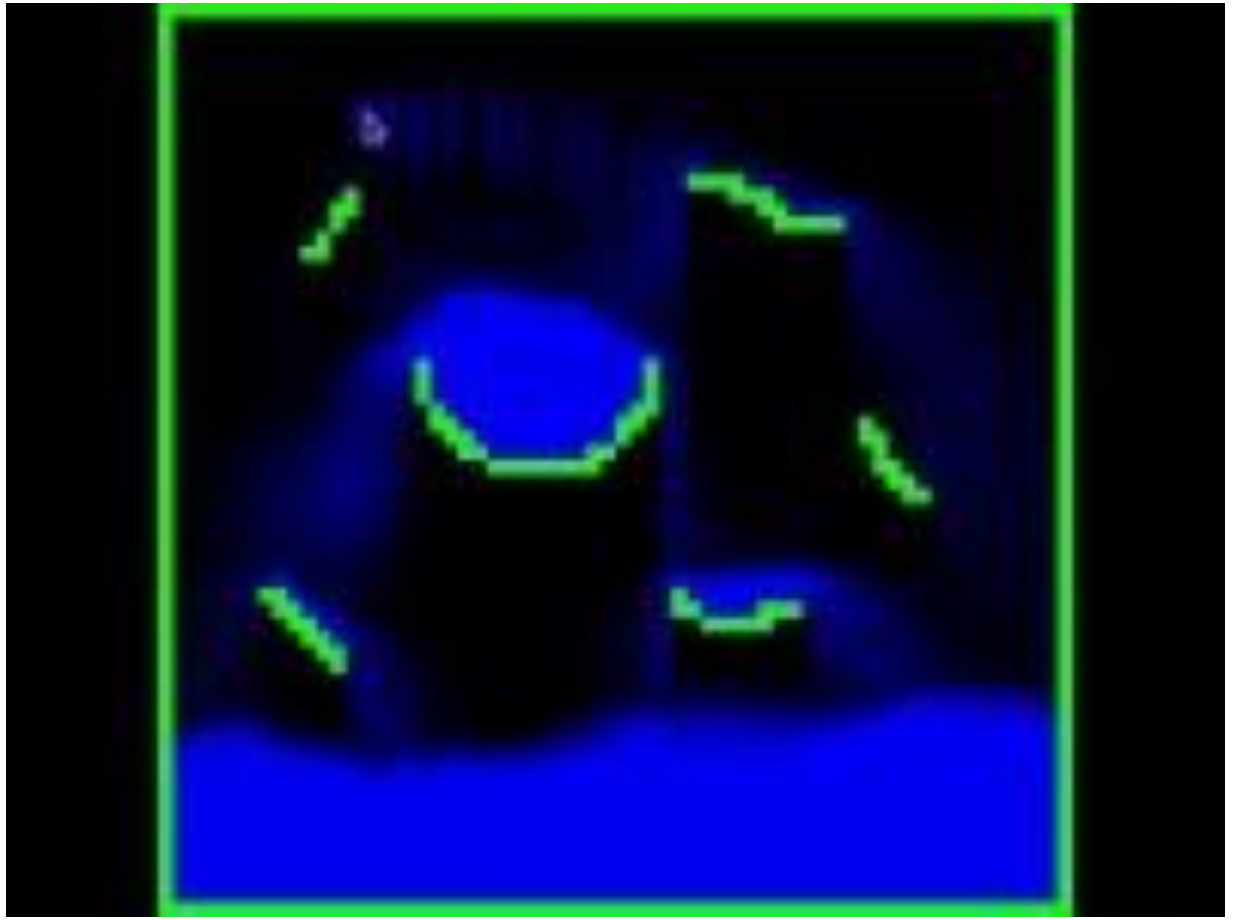
- Exactly like in the case of spatial simulations, we can use **periodic boundary conditions** to apply rules on the boundary

- The alternative would be to use a default value (e.g., 0) to fill the state of boundary cells having no neighbors



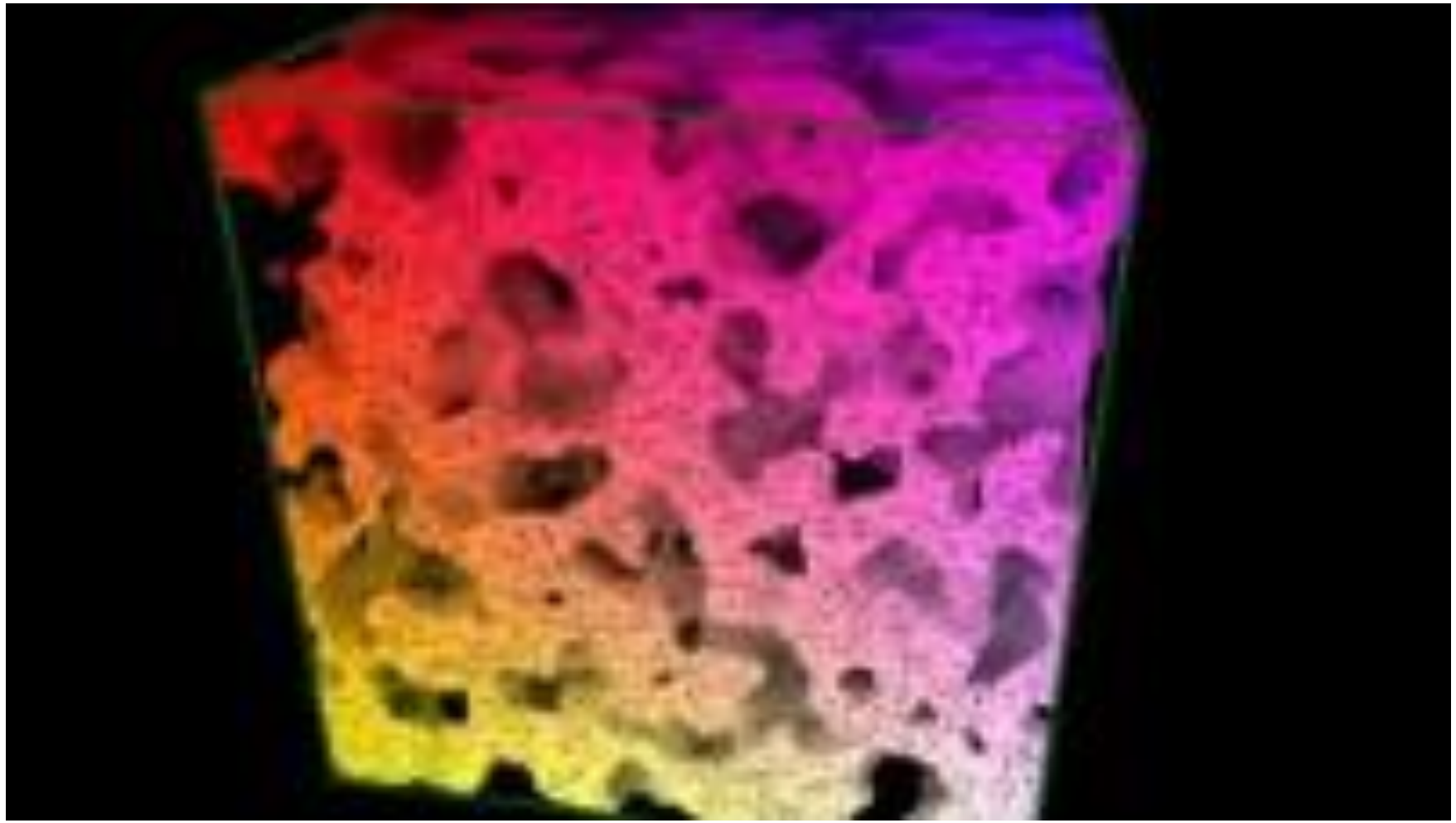


Fluid simulation with CAs



https://www.youtube.com/watch?v=XfhNlTt3zeQ

3D CA



https://www.youtube.com/watch?v=dQJ5aEsP6Fs

Agent-based modeling

- **Agent-Based Modelling** (ABM) uses software to **simulate the (local) behavior** of a group of agents, in order to determine the whole system's **emergent phenomena**

- **Each agent is *simple***, with limited capabilities
  - Each agent can have an **internal state**
  - The behavior of each agent is governed by **rules**
  - *"Macro-effects emerge from simple micro-rules"*

- Agents are **placed** in an **environment**
  - Agents **can sense** the environment
  - Agents **react** to what they sense

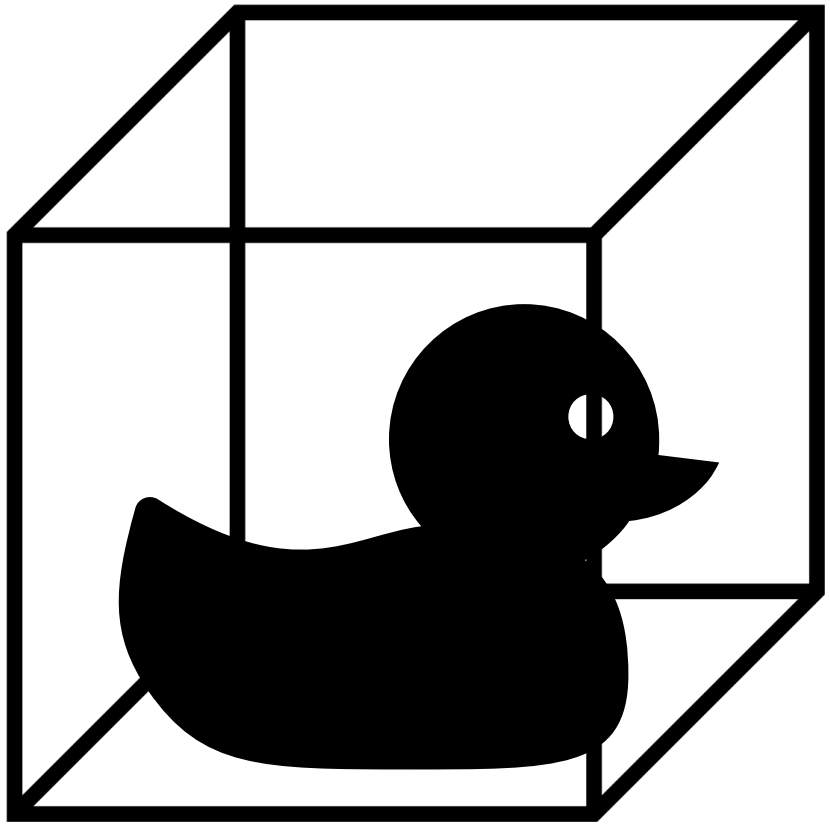
Agent's movement and collisions

- Agents are placed and move in an **environment**
  - There are **specific rules for movement**
  - Different agents can have different rules

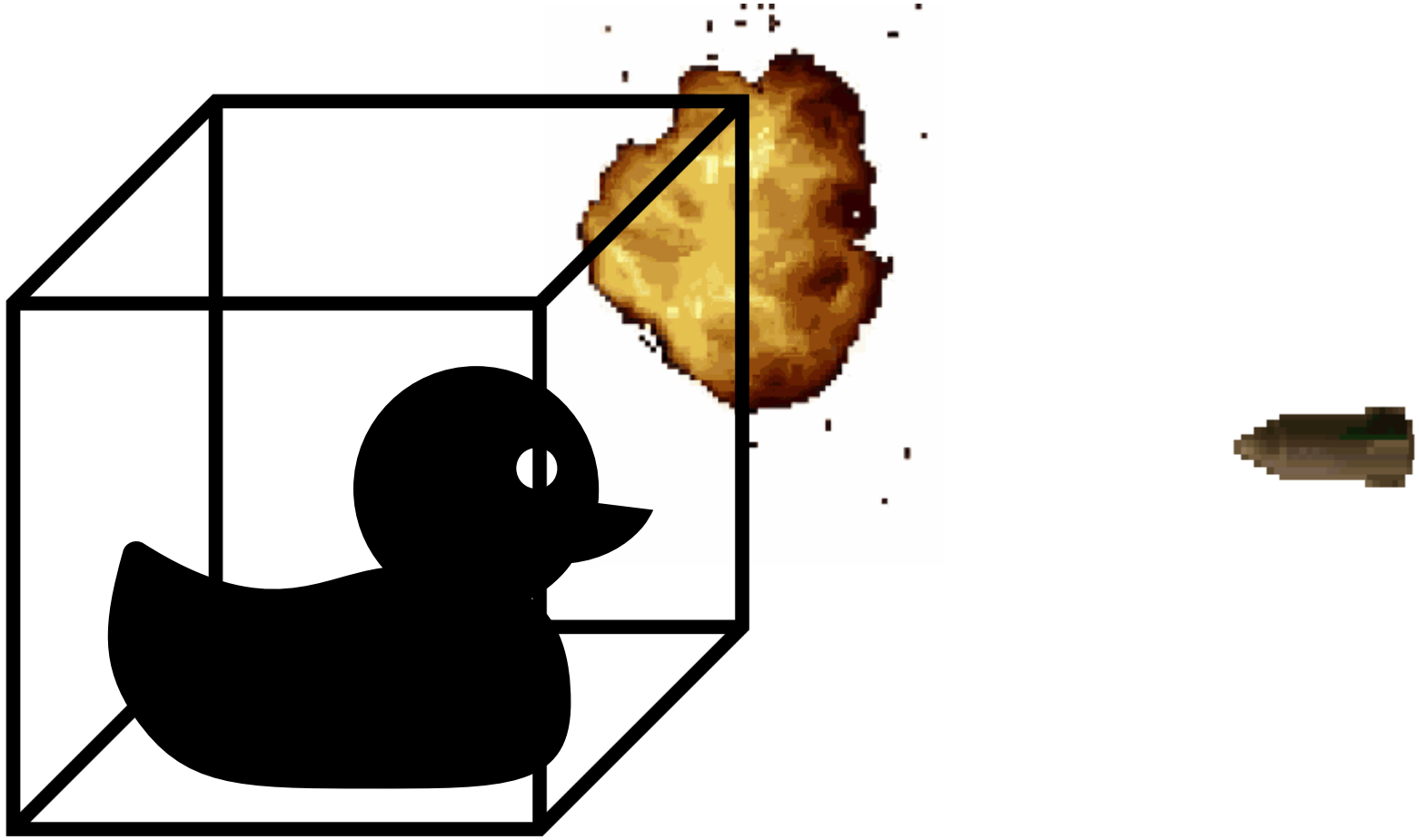
- Movement → plenty of occasions for **collisions**
  - With the **environment** itself (e.g., walls)
  - With **other agents** (e.g., reactions)

- At each iteration of the simulation, we must **check any possible collision**

Bounding boxes

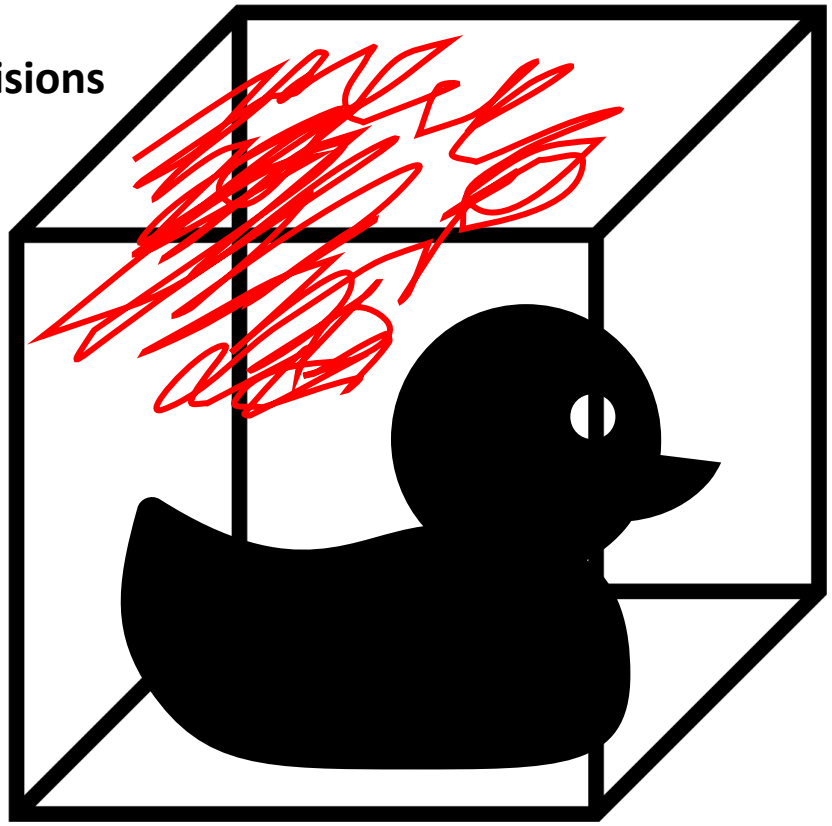


Bounding boxes



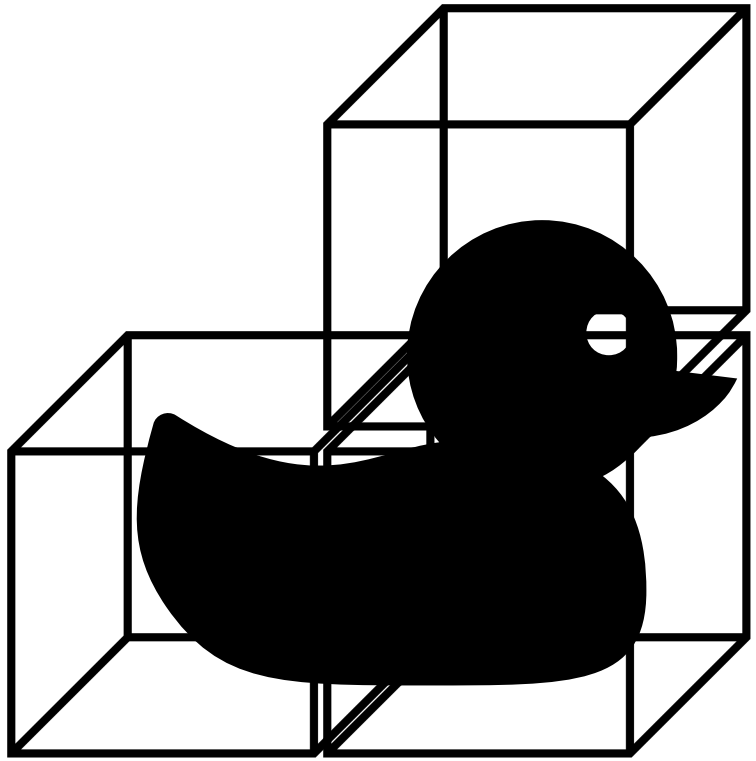
Bounding boxes

Region of **fake collisions**



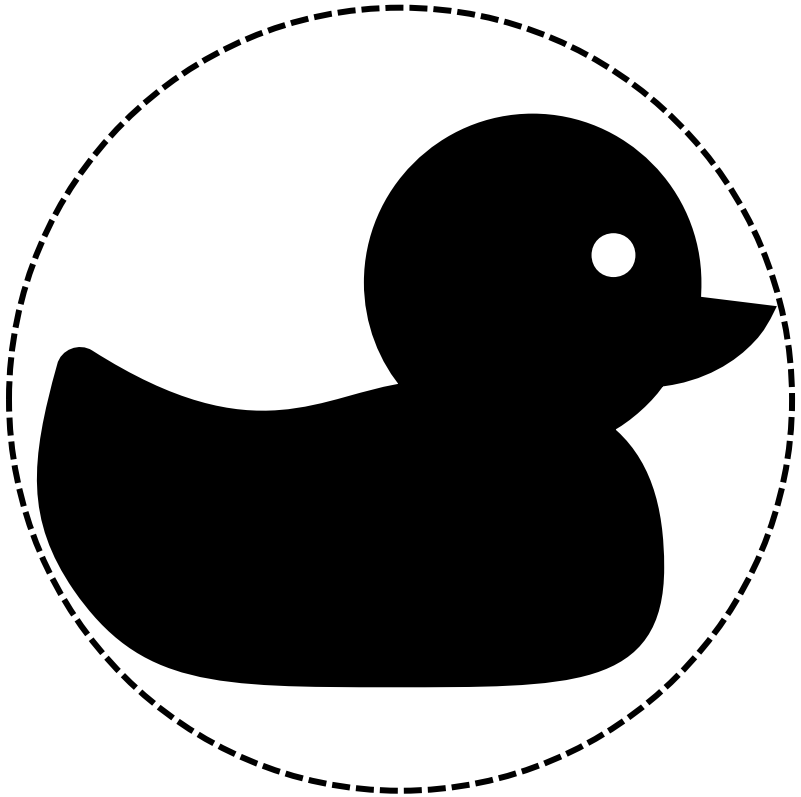
Multiple bounding boxes?

- Better, but increased calculations!



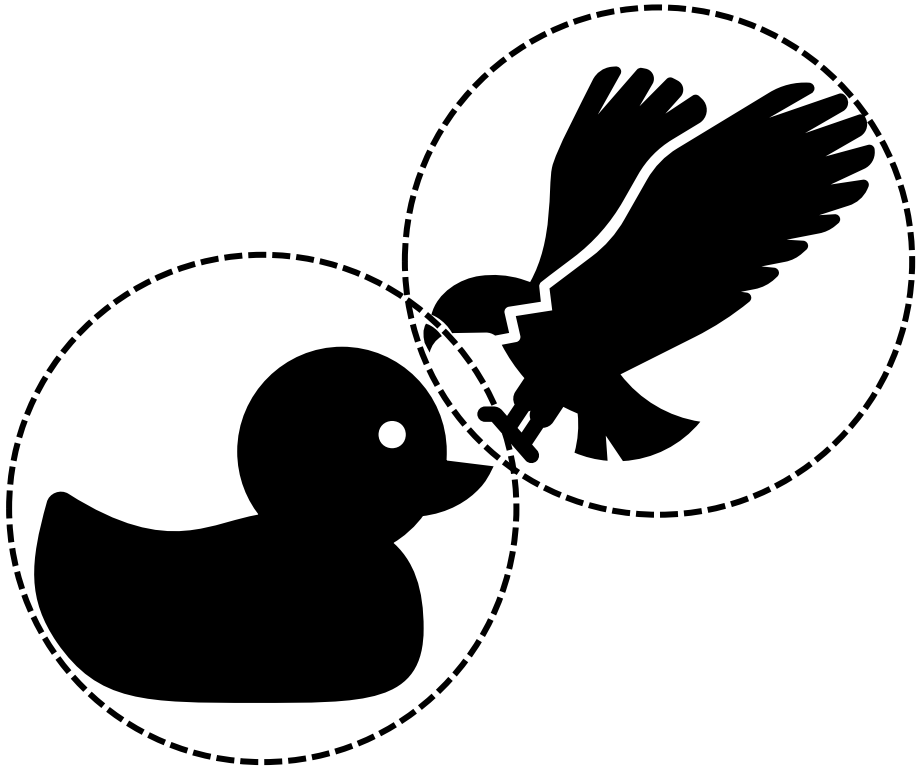
Bounding spheres?

- Better trade-off accuracy:calculations



Bounding spheres?

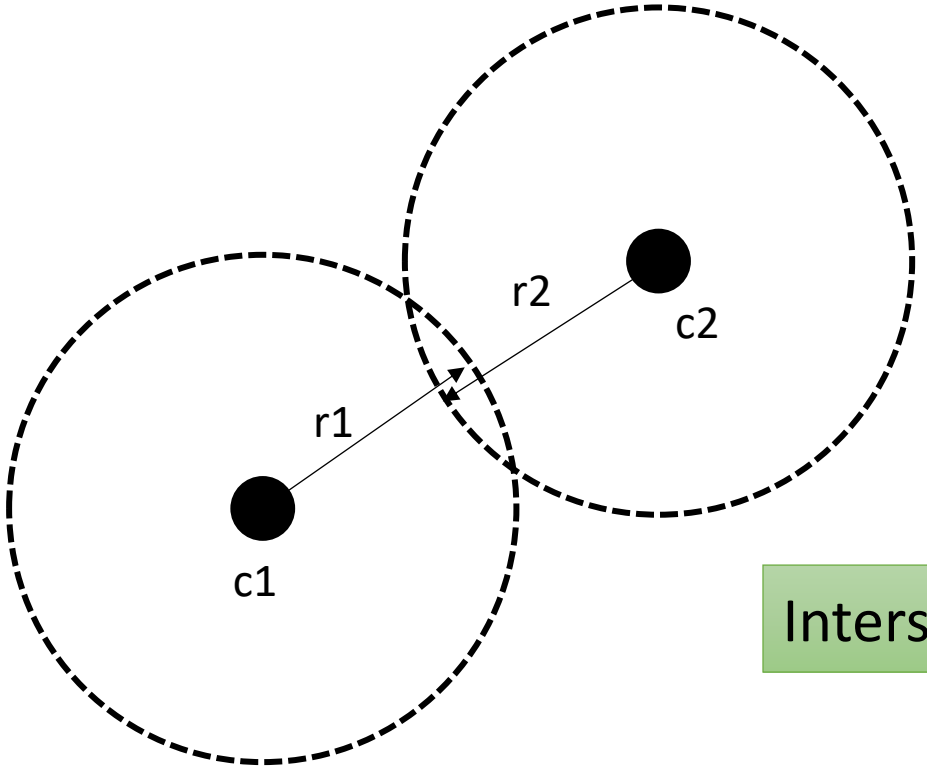
- Better trade-off accuracy:calculations



Collision between two agents → **intersection** between their spheres

Bounding spheres?

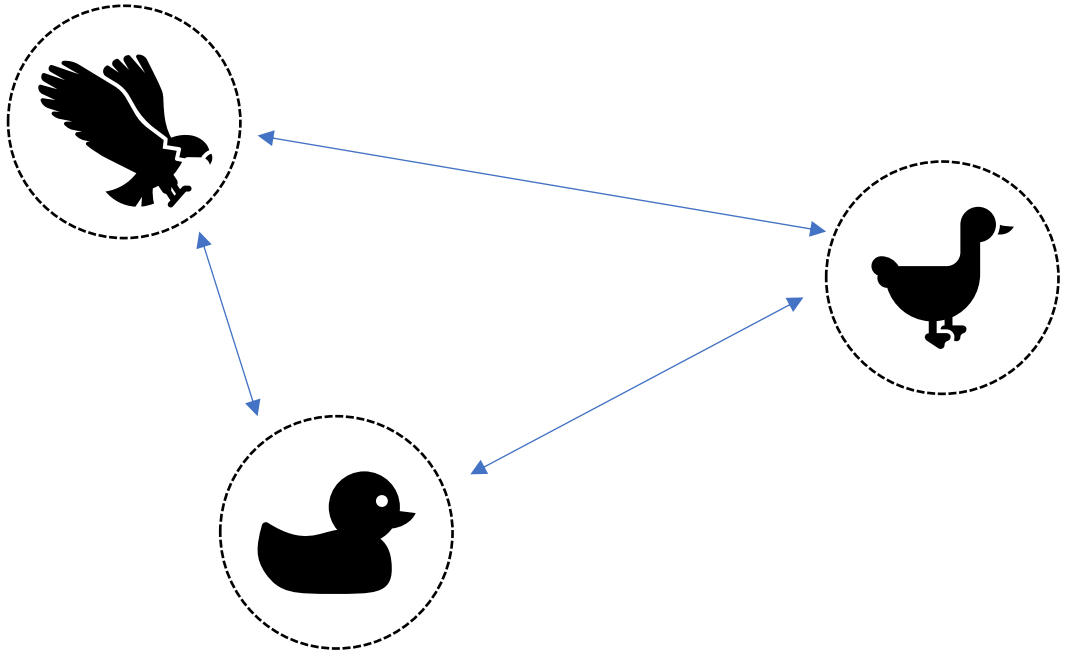
- Better trade-off accuracy:calculations



Collision between two agents → **intersection** between their spheres

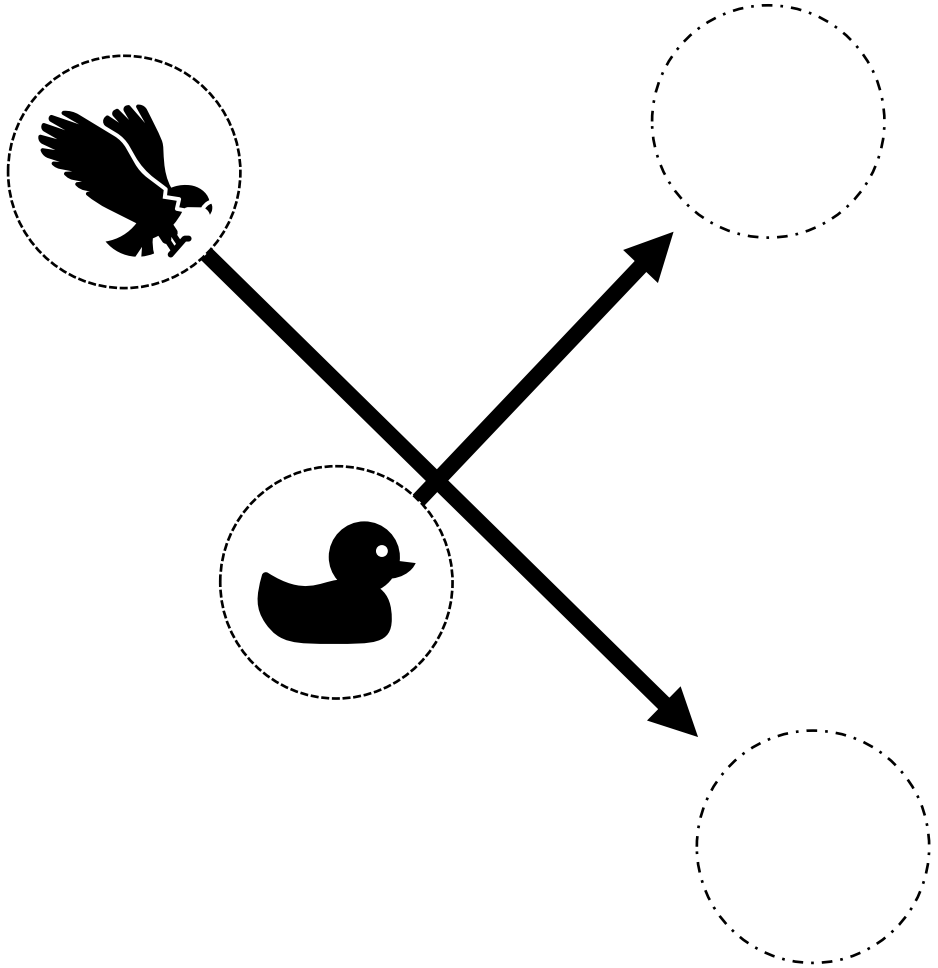
Intersection $\stackrel{\text{def}}{=} \left||c1 - c2|\right| < ||r1|| + ||r2||$

A lot of calculations



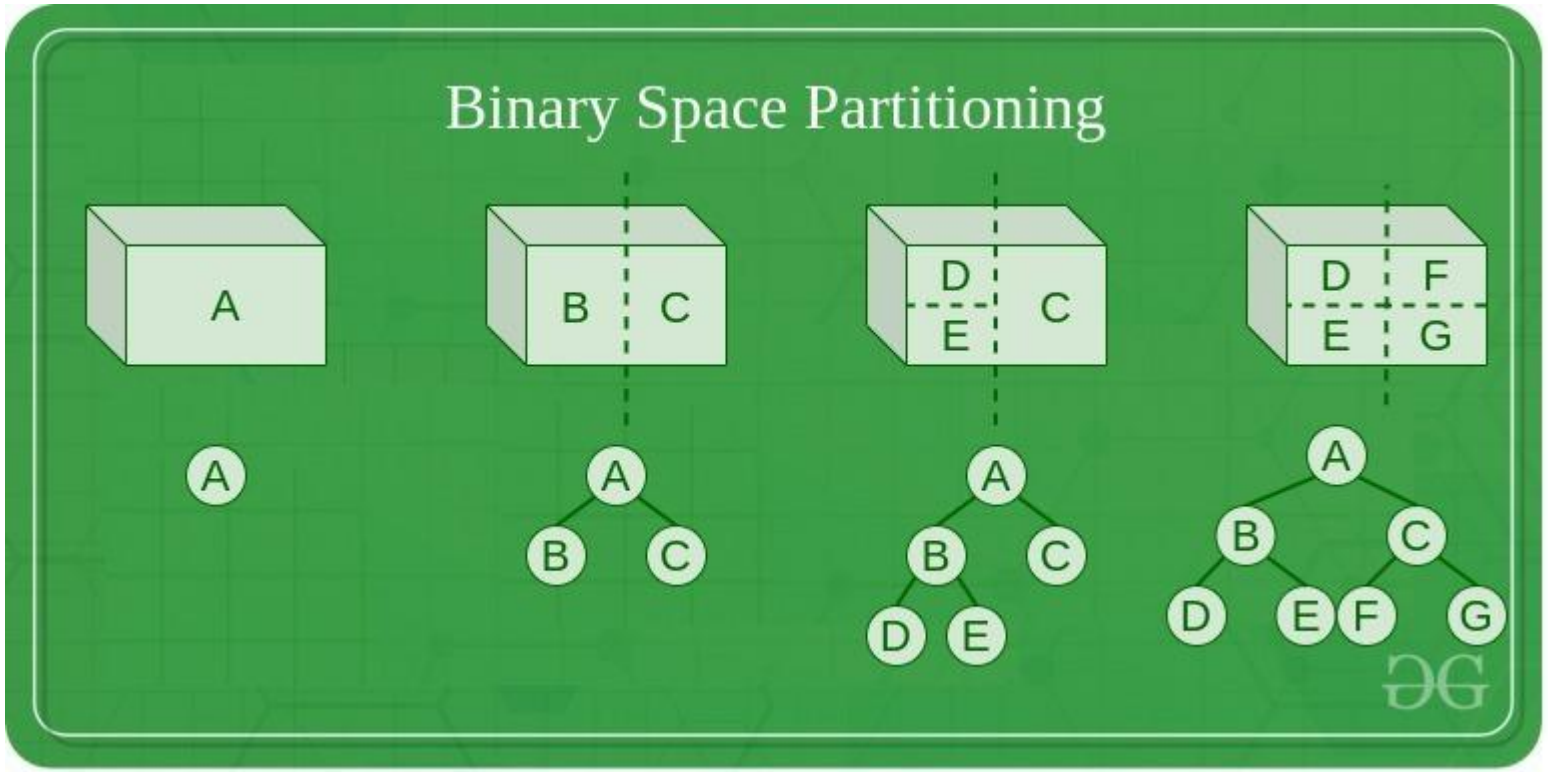
Given N agents, we have to calculate $\frac{N^2}{2}$ possible collisions for each time step

A LOT of calculations

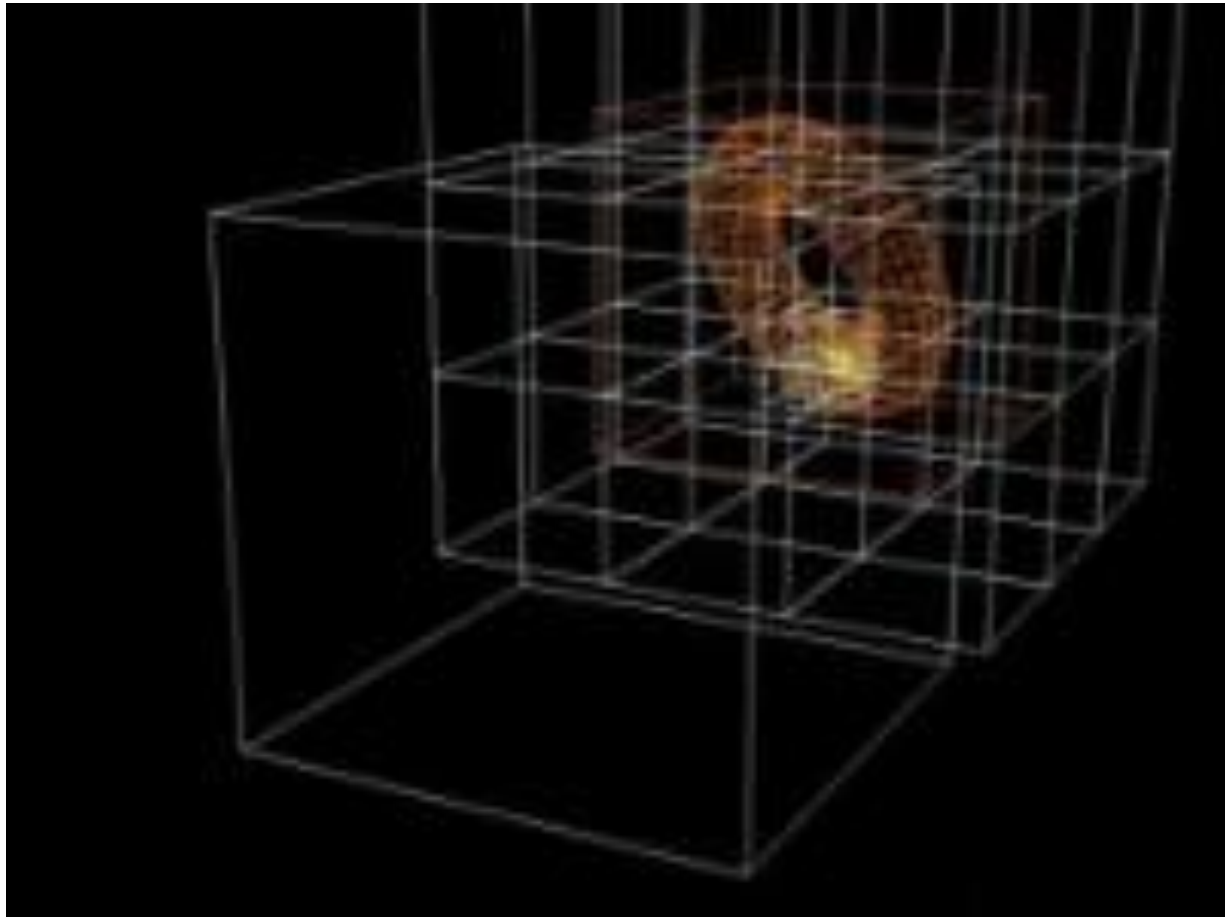


If the velocity of the agents is too high, some **collisions might be overlooked**!

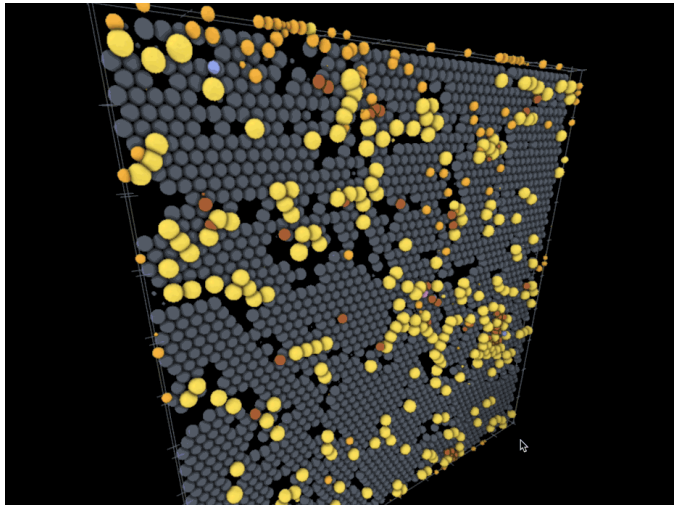
Binary space partitioning (2D)



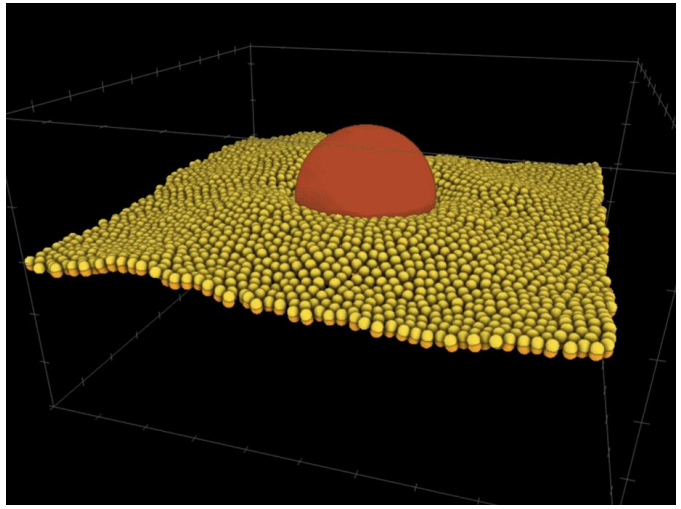
Space partitioning in 3D → octrees



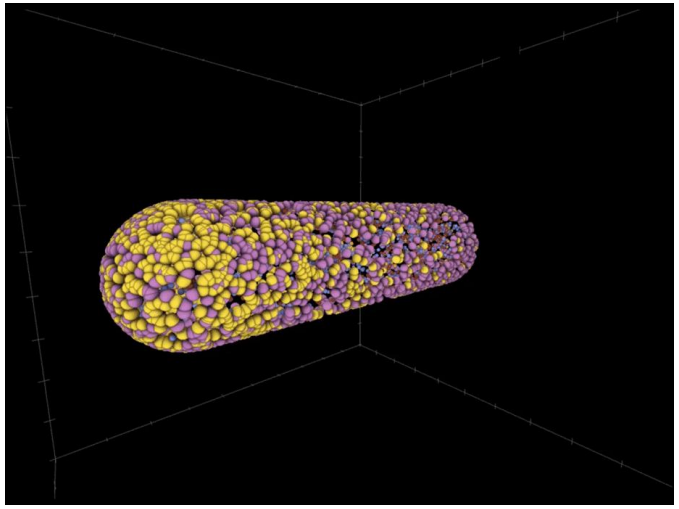
Agent-based simulation



SARS-CoV-2 Dynamics in Human Lung Epithelium



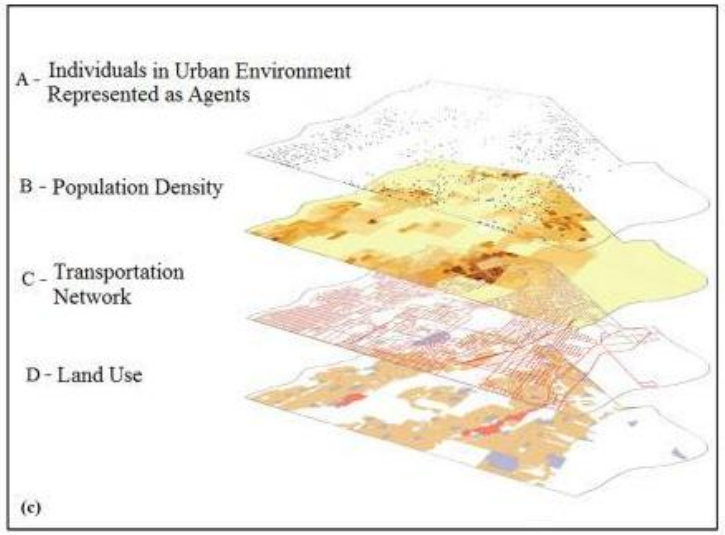
Membrane Wrapping a Nanoparticle

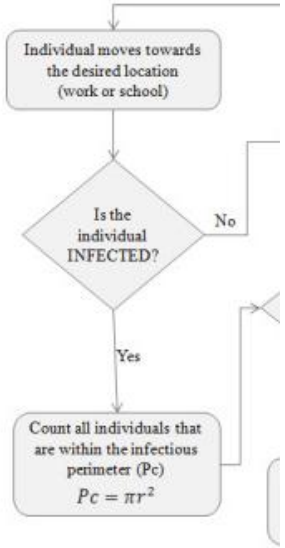


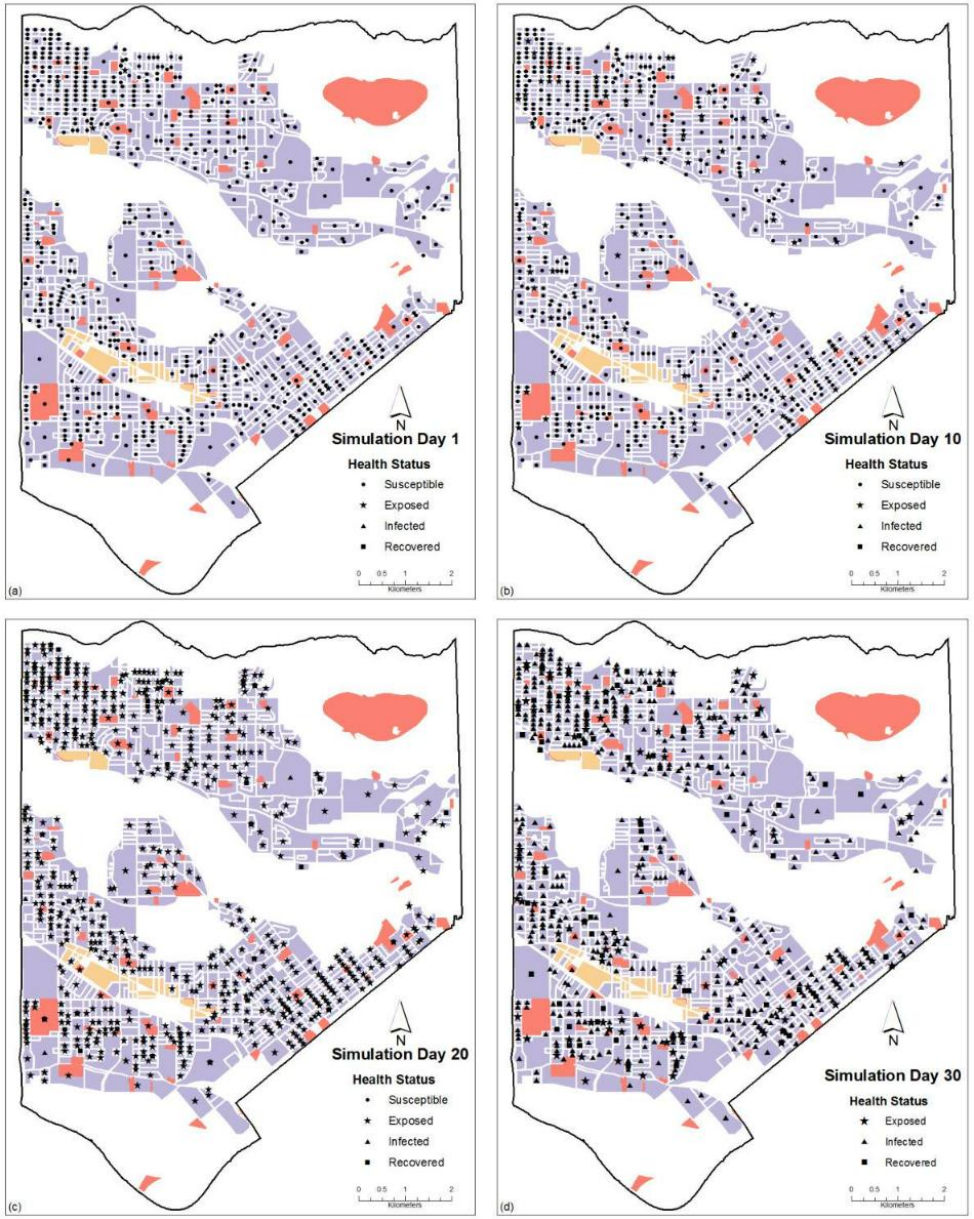
Spatiotemporal oscillations in the *E. coli* Min system

ABM for disease spread

- **ABM integrated with GIS** (geographic information)
  - Agents **move on a map**, **behavior** based on **statistical data**
  - Simulation of actual disease spread in **urban environment** (e.g., transportation network)
  - **SEIR model**: agents can be susceptible, exposed, infected and recovered
  - **Infection == contact** == collision
  - [Perez and Dragicevic , Int J Health Geogr 2009]



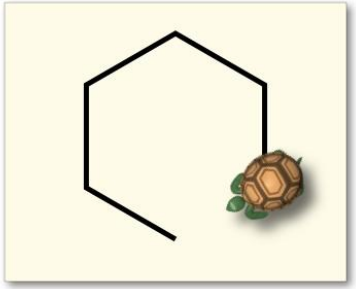




ABM in practice: NetLogo?

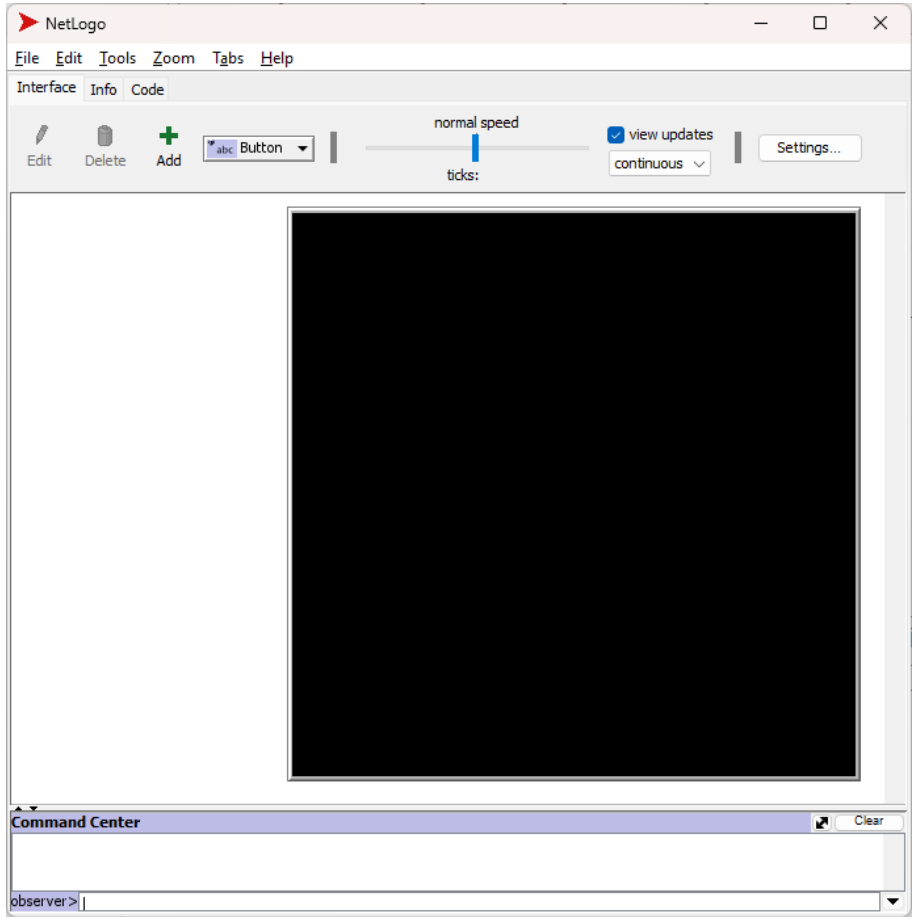
- NetLogo is a …

- Basic agent (and building block) in NetLogo: the **"turtle"**
  - You can give commands like "move forward 10, rotate 45°…"
  - Turtles have IDs, position, internal states, **properties**…
  - Turtles can **draw**: ask to "pen-down" or "pen-up"
  - Turtles move in a (continuous) **environment**

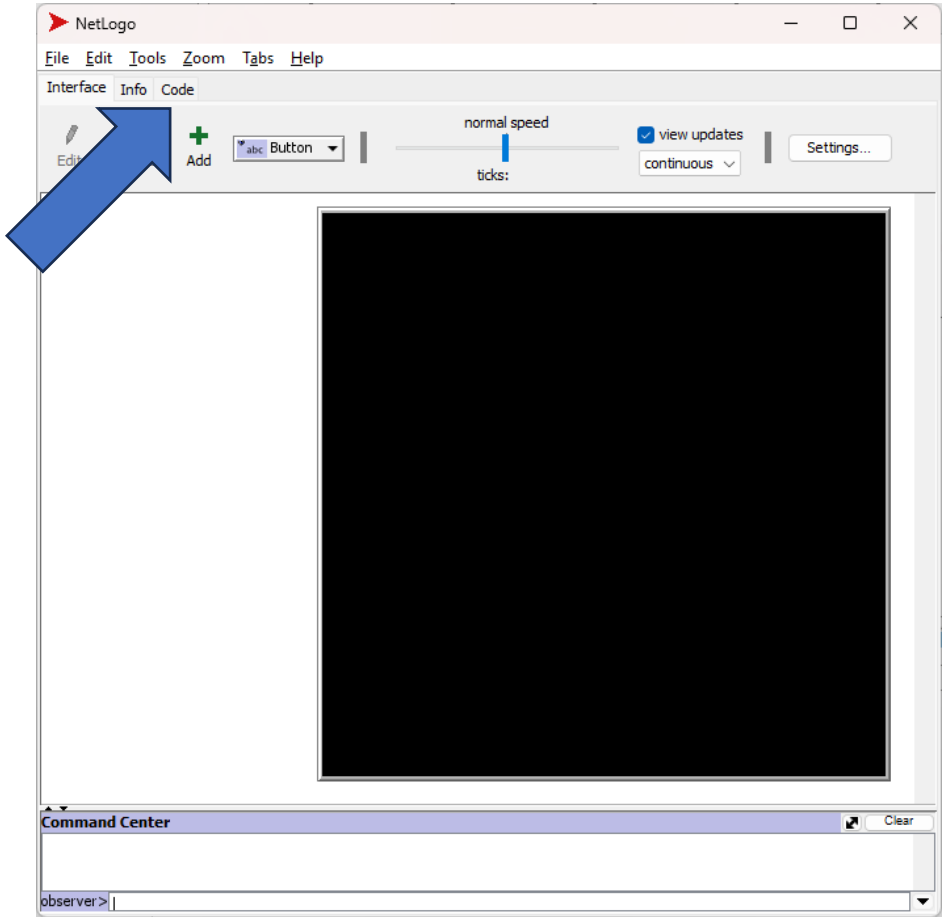


- You can **implement code** (using NetLogo's language)…

NetLogo's window



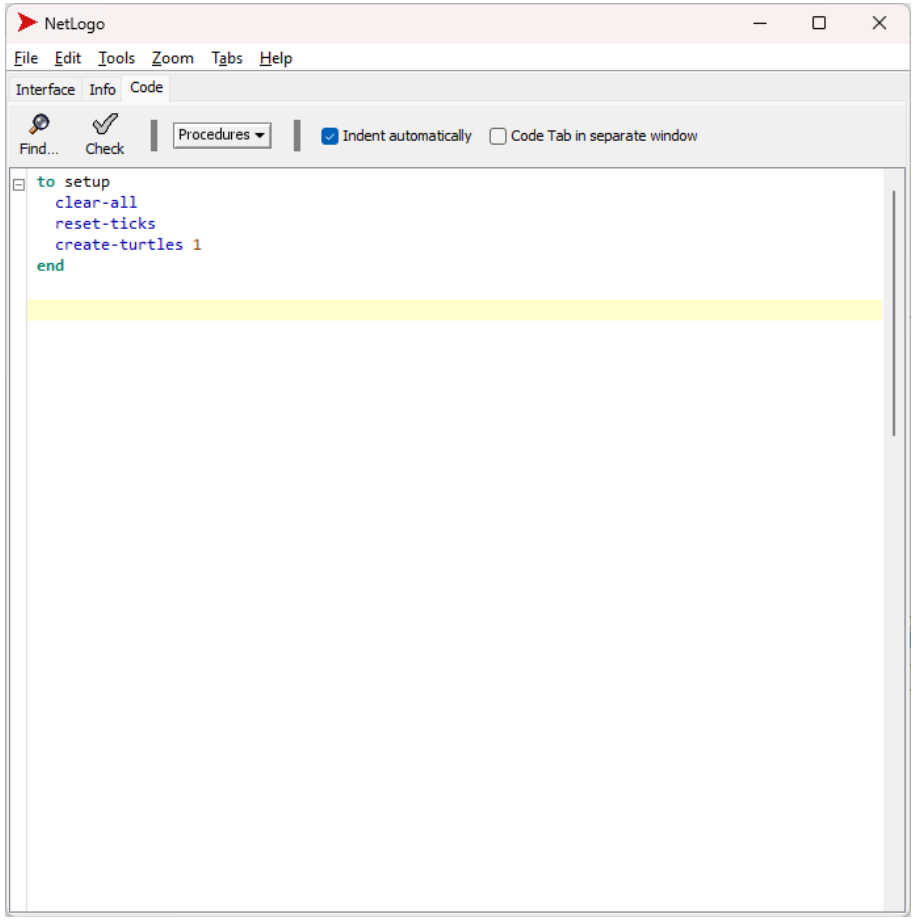
NetLogo's window



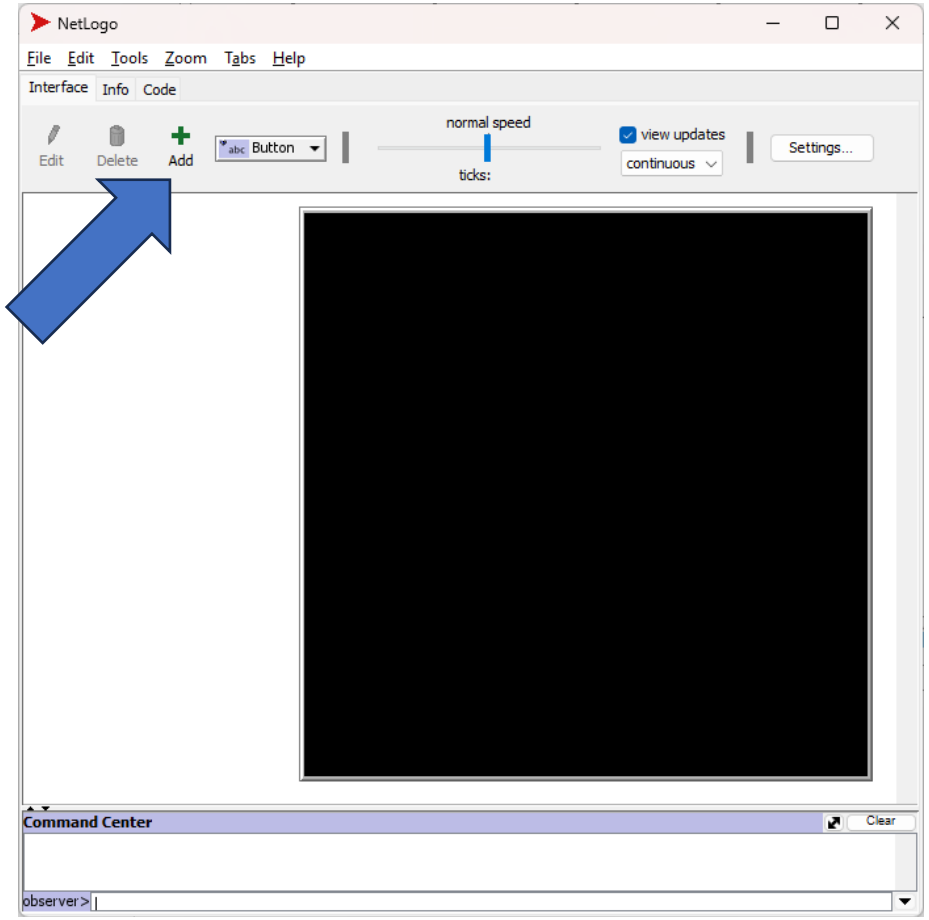
Entering code

In this example, I am creating a function that sets up the simulation:

1) clear everything
2) reset time to 0
3) create ONE agent/turtle



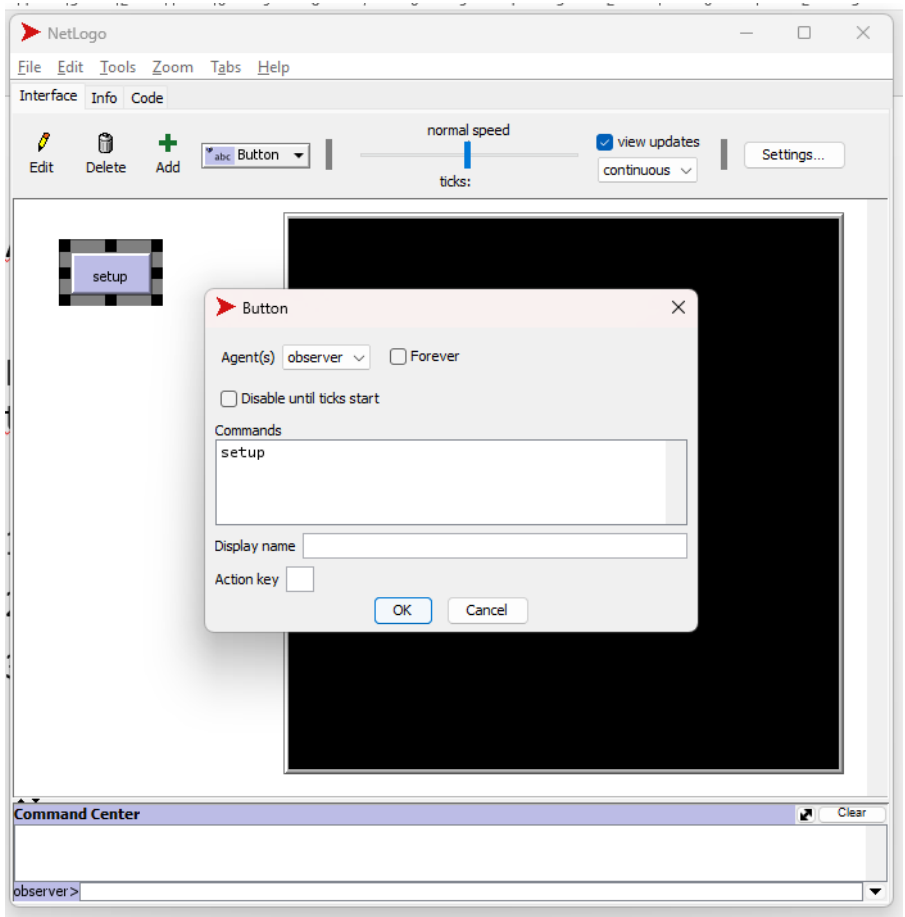
Creating a GUI for the simulator



Adding buttons

I added a **button**, associated to the "setup" function

Now, when I press that, it **resets the environment** to the initial conditions

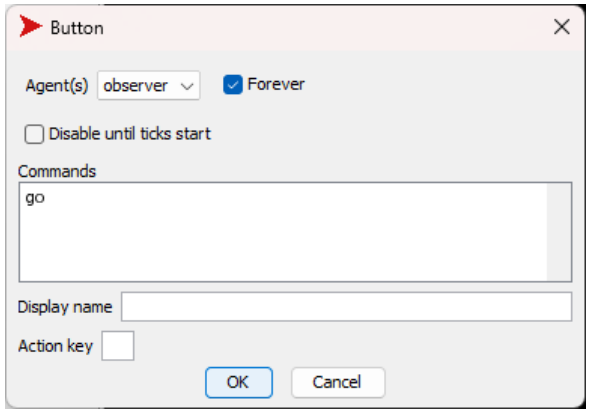


Implementing a behavior

- I can tell the turtles to move one **step forward** per tick and **rotate by a random angle** between -5 and +5

- I can use the "forever" option to make it an **infinite** loop

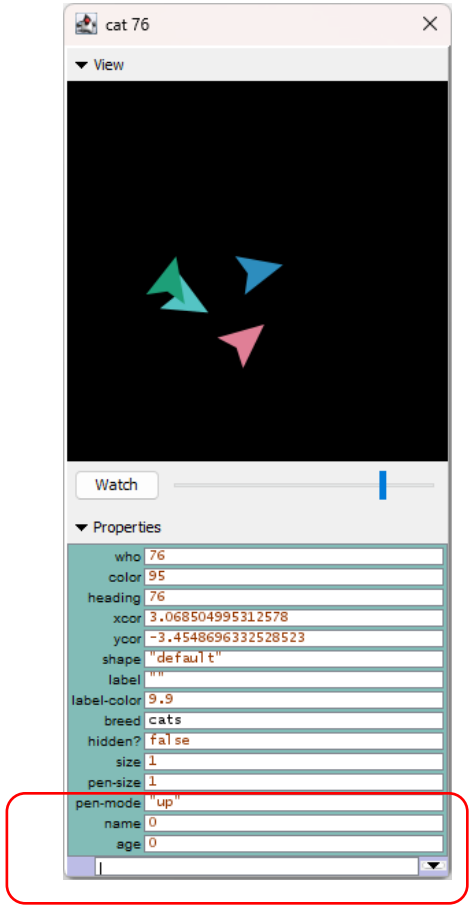
```
to setup
  clear-all
  reset-ticks
  create-turtles 200
end

to go
  ask turtles [
    forward .1
    set heading (heading - 5 + random 10)
  ]
end
```



Implementing multiple behaviors

- We can breed **species**. For instance, we can do:

```
breed [cats cat]

to setup
  clear-all
  reset-ticks
  create-turtles 200
end

to go
  ask turtles [
    forward .1
    set heading (heading - (angle / 2) + random angle)
  ]

  ask cats [
    forward .01
  ]
end
```

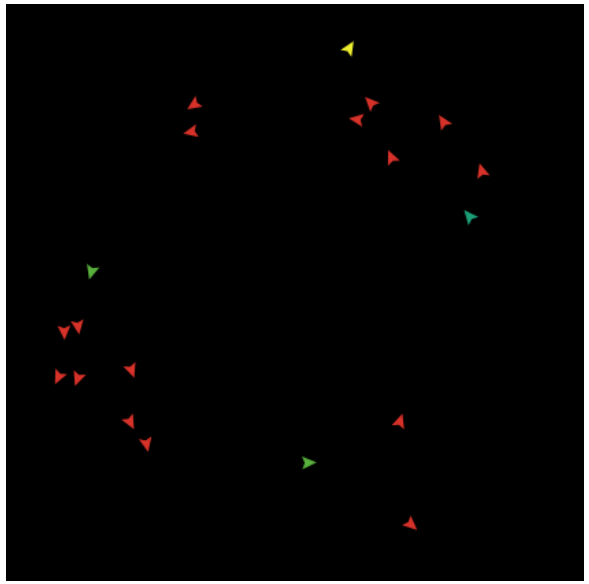


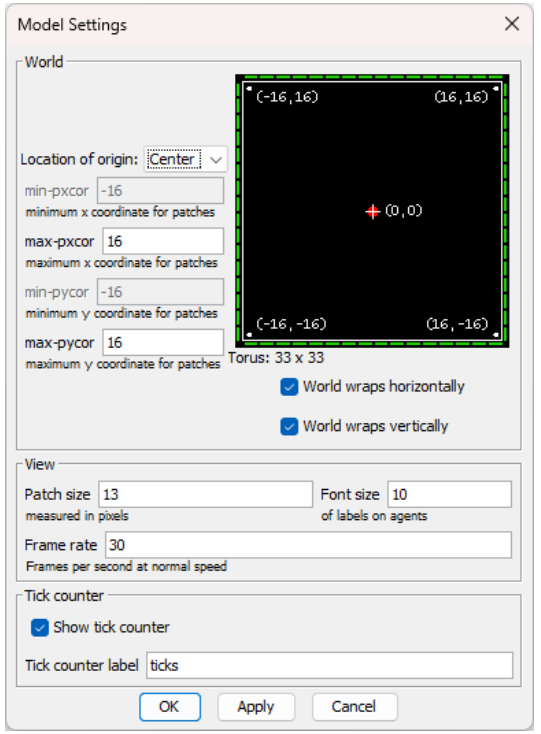
- We can use the syntax –own to give attributes to breeds like: **cats-own [name age]**

Patches

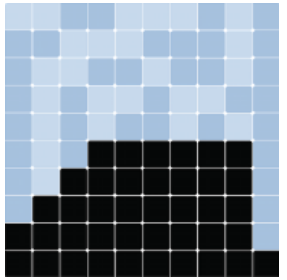
- The environment is subdivided in **patches**
- Two agents collide when they **occupy the same patch**

```
breed [cats cat]

to setup
  clear-all
  reset-ticks
  create-cats 20
  ask cats [forward 5]
end

to go
  ask cats [
    forward .05
    set heading (heading - (angle / 2) + random angle)
  ]

  ;; here it is
  ask cats [ check-for-collision ]

end

to check-for-collision

  if count other cats-here = 1
  [
    set color red
  ]
end
```





Another option: Mesa



- Mesa is a Python library for agent based modeling
- Base objects: **mesa.Agent**, **mesa.Model**
- Agents can move in different ways (e.g., on **spatial grids**)
- The Model exploits a **scheduler** to harmonize and perform **simulation steps**
- Check out Mesa's tutorial here: https://mesa.readthedocs.io/stable/tutorials/intro_tutorial.html

One example with MESA: housing market

- The model uses agents to simulate a housing market https://medium.com/@ptlabadie/agent-based-models-in-python-simulating-a-housing-market-94a30be84924

- Inspiration was taken from [Axtell et al. 2014], which models the Washington D.C. housing market bubble, and [Baptista et al. 2016], which looks at the UK housing market

- The goal was to **determine the impact of initial parameters** to the outcome

Agents

Agents are mainly people (Households) who start off **displaced with no housing**

- They **must rent or buy**
- Households **start with some wealth** and earn **income**
- Households **try to buy** houses if they can **afford them without credit**
- If households are unable to purchase a house, they will **try to rent**
- Rental: **up to 30%** of income
- The **rental** supply is created by **households owning more than one** house (“investors”)
- These houses are placed on the **rental market**

Let’s give a look at the classes…

Household agent object


```
class Household(mesa.Agent):
    """An agent with wealth, income and housing needs"""
    def __init__(self, unique_id, model, income_avg,income_std,wealth_avg,wealth_s
        super().__init__(unique_id, model)

        self.wealth = np.random.normal(wealth_avg,wealth_std)
        # Only positive incomes
        self.income = abs(np.random.normal(income_avg,income_std))
        self.agent_type = "displaced"
        self.houses = []
        self.rent_payment = None
        self.leased_house = None
```

Household: some methods

```
def buy_house(self):
    """Buy a house and add it to the list of owned houses"""
    for house in housing_stock:
        if house.status == "vacant" and house.price < self.wealth:
            house.status = "owned"
            house.owner = self.unique_id
            self.wealth -= house.price
            self.houses.append(house.id)
            break
```

```
def rent_out_house(self):
    """If you have multiple houses, rent them out"""
    for house in housing_stock:
        # Match additional house to housing stock
        if house.id == self.houses[-1]:
            house.status = "for_rent"

def collect_rent(self):
    """If you have rented houses, collect the rent payment"""
    if self.agent_type == "investor":
        for house in housing_stock:
            if house.status == "rented" and house.id in self.houses:
                self.wealth += house.rent
```

```
def rent_house(self):
    """Rent a house"""
    for house in housing_stock:
        if house.status == "for_rent" and house.rent < 0.3*self.income:
            house.status = "rented"
            self.agent_type = "renter"
            self.rent_payment = house.rent
            self.leased_house = house.id
```

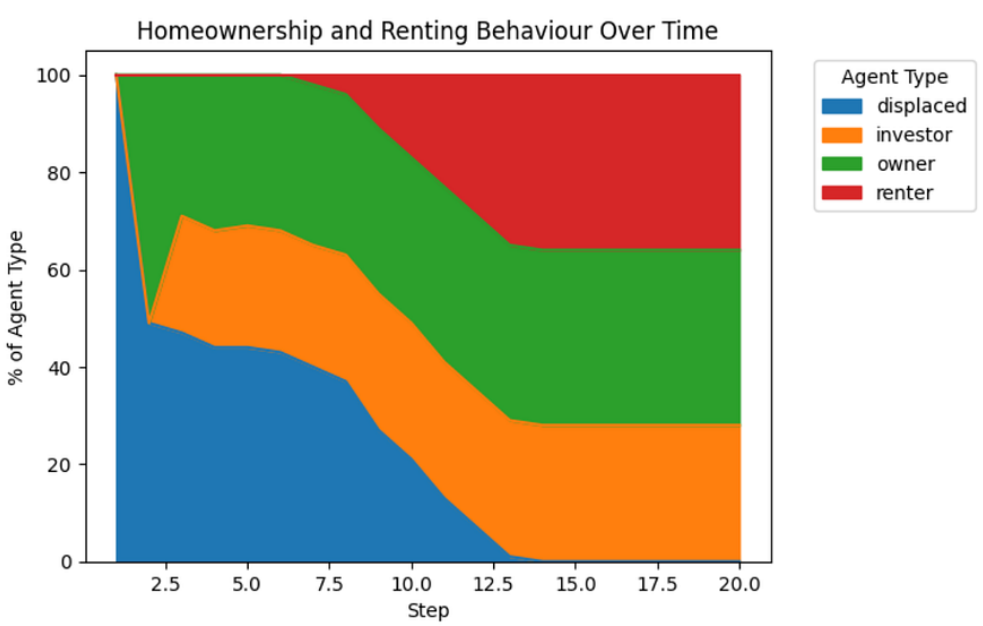
```
def step(self):
    print(f"ID#: {self.unique_id} is a {self.agent_type} with income: {self.
    # Generate income every year
    self.wealth += self.income

    self.income = self.income*1.05
    # Determine agent type
    if len(self.houses) > 1:
        self.agent_type = "investor"
        # Rent house if you have multilple
        self.rent_out_house()

    if len(self.houses) == 1:
        self.agent_type = "owner"

    # Try to buy a house
    self.buy_house()

    # If renting, remove rent from wealth
    if self.agent_type == "renter":
        self.wealth -= self.rent_payment

    # If too expensive, try to rent a house
    if self.agent_type == "displaced":
        self.rent_house()
```

Houses and the Model of housing market

```python
class House():
    """A class representing a house asset."""
    next_id = 1
    def __init__(self,h_price, h_price_std, h_rent,h_rent_std):
        # Assign the next available id to the current instance
        self.id = House.next_id
        # Increment the next available id for the next instance
        House.next_id += 1
        self.price = np.random.normal(h_price,h_price_std)
        self.quality = np.random.uniform(1,10)
        self.rent = np.random.normal(h_rent,h_rent_std)
        self.owner = None
        self.status = "vacant"
```

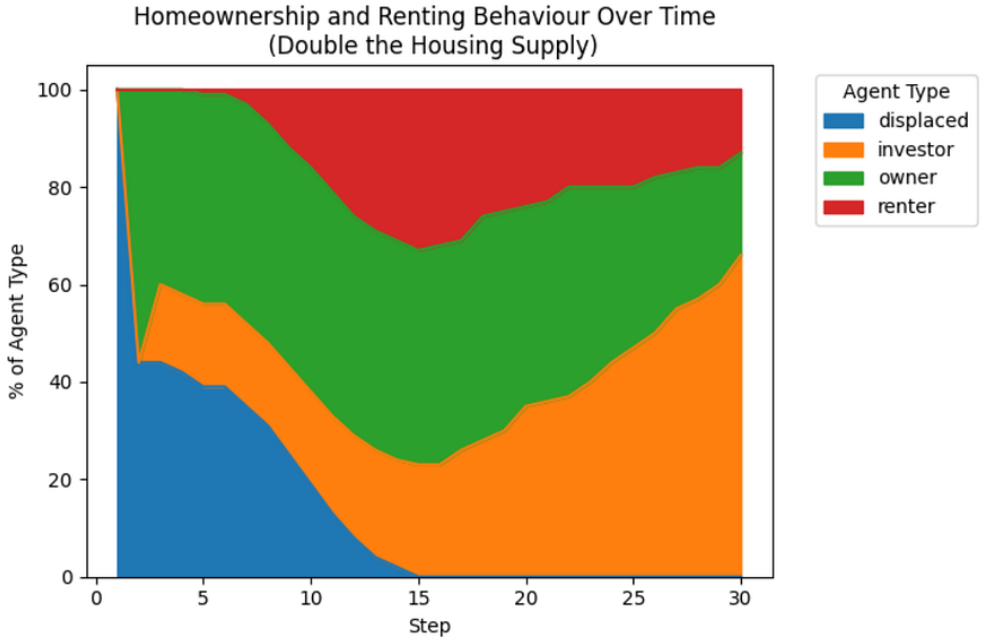
```python
class HousingMarket(mesa.Model):
    """Model with Household Agents"""
    def __init__(self, N,income_avg,income_std,wealth_avg,wealth_std):
        self.num_agents = N
        self.schedule = mesa.time.RandomActivation(self)
        self.income_avg = income_avg
        self.income_std = income_std
        self.wealth_avg = wealth_avg
        self.wealth_std = wealth_std

       # Custom agent reporter function to calculate the length of the 'houses'
        def count_houses(agent):
            return len(agent.houses)

        # Initialize DataCollector with the custom agent reporter
        self.datacollector = DataCollector(agent_reporters={"Wealth": "wealth",

        # Create agents
        for i in range(self.num_agents):
            a = Household(i, self,self.income_avg,self.income_std,self.wealth_av
            self.schedule.add(a)

    def step(self):
        # Call moodel and move it one period forward
        self.schedule.step()
        # Collect agent-level data at each step
        self.datacollector.collect(self)
```

Some results…



The model predicts that, eventually, **all houses are sold**

This is due to the fact that the model's **parameters are constant** and there is **no pressure whatsoever to sell**



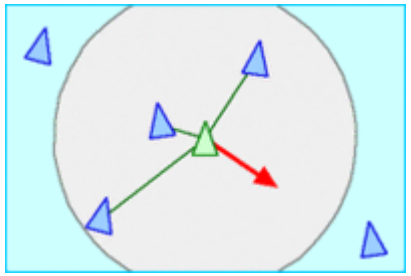
One simple modification to the number of houses shows that **doubling the available houses leads to less renters**

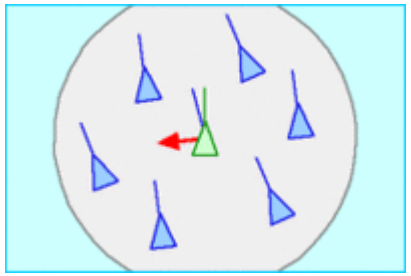
BOIDS

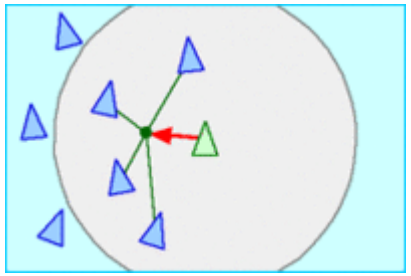
https://www.youtube.com/watch?v=9iDA6WMqEyQ

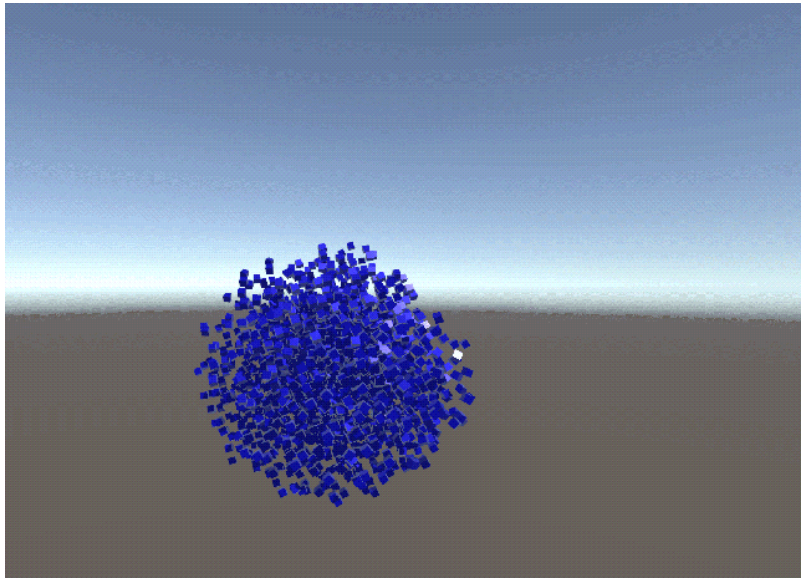
BOIDS: swarms emerge from just 3 rules

- [Reynolds 1986]
- **Separation** (collision avoidance)
- **Alignment** (uniform to neighbors)
- **Cohesion** (don't fly solo)









- Trivia: BOIDS have been used to model bats in Batman movies

Swarm's emerging behavior

- Swarms are fascinating **complex systems**
- Also from a Physics point of view. See, e.g., the work of Nobel prize recipient [Parisi *et al*., PNAS 2010]

Swarm Intelligence

- Swarms can solve complex problems → **Swarm Intelligence**, forthcoming topic!

Summary

- Cellular Automata are a computational model based on "cells" updating using a massively parallel approach based on simple local rules
- Agent-Based Simulation exploits agents located in a simulated environment
- In both cases, a complex behavior can emerge from very simple "local" rules